

The Concise TypeScript Book

Simone Poggiali

الموجز TypeScript كتاب

الموجز نظرة شاملة ومختصرة على إمكانيات TypeScript يقدم كتاب ويقدم شروحات واضحة تغطي جميع جوانب أحدث إصدار من TypeScript. اللغة، بدءًا من نظام الأنواع القوي وصولًا إلى الميزات المتقدمة.

سواء أكنت مبتدئًا أم مطورًا متمرسًا، فإن هذا الكتاب مورد قيم للغاية لتعزيز TypeScript فهمك وإتقانك للغة.

هذا الكتاب مجاني بالكامل ومفتوح المصدر.

أؤمن بأن التعليم التقني عالي الجودة ينبغي أن يكون متاحًا للجميع. ولهذا السبب، أبقى الكتاب متاحًا مجانيًا وأحدثته بانتظام بإدخال تحسينات وأمثلة جديدة.

الموجز TypeScript من كتاب Plus Edition اكتشف الإصدار.

The Concise TypeScript Book

Plus Edition

React and Real-World Patterns
For **TypeScript 7**

Simone Poggiali

بالنسبة إلى القراء الذين يريدون تجاوز الإصدار المفتوح المصدر، يتضمن **أنماط React: الموجز TypeScript من كتاب Plus Edition الإصدار** محتوى إضافيًا وحصريًا يركّز على التطبيق العملي **TypeScript 7 واقعية** لـ.

ما يلي Plus Edition يتضمن الإصدار:

- **TypeScript تغطية لأحدث ميزات — TypeScript 7 محدّث من أجل** والتحسينات التي أدخلت على اللغة 7.
- **إرشادات عملية لتعريف أنواع المكوّنات، — React مع TypeScript** وأنماط `refs`، `children` والأحداث، و `hooks`، و `props` الشائعة.
- **للمشروعات الواقعية — أمثلة مركّزة تعالج TypeScript وصفات** المشكلات العملية التي يواجهها المطوّرون عند بناء تطبيقات

وصياتها TypeScript.

فإنك تدعم مباشرةً أيضًا استمرار Plus Edition من خلال شراء الإصدار تطوير الكتاب المجاني مفتوح المصدر وصيانتها.

Amazon باللغتين الإنجليزية والإيطالية على Plus Edition يتوفر الإصدار [واشتره من Plus Edition](#) في جميع أنحاء العالم. [استكشف الإصدار Amazon](#).

دعم المشروع

إذا ساعدك الكتاب المجاني على إصلاح خطأ برمجي، أو فهم مفهوم صعب، أو التقدّم في مسيرتك المهنية، فيُرجى التفكير في دعم عملي بدفع المبلغ الذي تريده، مع مساهمة مقترحة قدرها \$5، أو برعاية فنجان قهوة.

يساعدني دعمك على إبقاء المحتوى محدثًا وتوسيعه بأمثلة جديدة، وشرحات أوضح، وإرشادات عملية إضافية.



BUY ME A COFFEE

Donate PayPal

الترجمات

تُرجم هذا الكتاب إلى عدة لغات، منها

[الإنجليزية](#)

[البulgارية](#)

[الألمانية](#)

[الفرنسية](#)

[الإنجليزية](#)

[الإيطالية](#)

[اليابانية](#)

[الكورية](#)

[البلندية](#)

[\(البرتغالية\) البرازيل](#)

[السويدية](#)

[التركية](#)

[الفيتنامية](#)

[الصينية](#)

[الإسبانية](#)

[التايلاندية](#)

[الروسية](#)

[العربية](#)

التنزيلات والموقع الإلكتروني

EPUB: يمكنك أيضًا تنزيل إصدار

<https://github.com/gibbok/typescript-book/tree/main/downloads>

:يتوفر إصدار عبر الإنترنت على

<https://gibbok.github.io/typescript-book>

جدول المحتويات

- [الموجز TypeScript كتاب](#)
 - [دعم المشروع](#)
 - [الترجمات](#)
 - [التنزيلات والموقع الإلكتروني](#)
 - [جدول المحتويات](#)
 - [مقدمة](#)
 - [عن المؤلف](#)
 - [TypeScript مقدمة إلى](#)
 - [TypeScript؟ ما](#)
 - [TypeScript؟ لماذا](#)
 - [TypeScript وJavaScript](#)
 - [TypeScript توليد الشيفرة في](#)
 - [\(الحديثة الآن\) التحويل إلى إصدار أقدم JavaScript](#)
 - [TypeScript بدء استخدام](#)
 - [التثبيت](#)
 - [التهيئة](#)
 - [TypeScript ملف تهيئة](#)
 - [target](#)
 - [lib](#)
 - [strict](#)
 - [module](#)
 - [moduleResolution](#)
 - [esModuleInterop](#)
 - [jsx](#)
 - [skipLibCheck](#)
 - [files](#)
 - [include](#)

- [exclude](#)
- [importHelpers](#)
- [TypeScript نصائح للانتقال إلى](#)
- [استكشاف نظام الأنواع](#)
 - [TypeScript خدمة لغة](#)
 - [التنميط البينيوي](#)
 - [TypeScript قواعد المقارنة الأساسية في](#)
 - [الأنواع. يوصفها مجموعات](#)
 - [إسناد نوع: تصريحات الأنواع. وتوكيدات الأنواع](#)
 - [تصريح النوع](#)
 - [توكيد النوع](#)
 - [التصريحات المحيطة](#)
 - [فحص الخصائص. وفحص الخصائص الزائدة](#)
 - [الأنواع الضعيفة](#)
 - [\(الفحص الصارم للكائنات الحرفية\) \(الحدثة\)](#)
 - [استدلال الأنواع](#)
 - [استدلالات أكثر تقدمًا](#)
 - [توسيع النوع](#)
 - [Const](#)
 - [على معاملات الأنواع Const معدّل](#)
 - [Const توكيد](#)
 - [التعليق التوضيحي الصريح للنوع](#)
 - [تضييق النوع](#)
 - [الشروط](#)
 - [طرح استثناء أو الإرجاع](#)
 - [الاتحاد المميّز](#)
 - [حراس الأنواع المعرّفة من قبل المستخدم](#)
 - [Switch-true التضييق باستخدام](#)
- [الأنواع الأولية](#)
 - [string](#)
 - [boolean](#)
 - [number](#)

- [bigint](#)
- [Symbol](#)
- [null](#) و [undefined](#)
- [Array](#)
- [any](#)
- [التعليقات التوضيحية للأنواع](#)
- [الخصائص الاختيارية](#)
- [الخصائص المخصصة للقراءة فقط](#)
- [توابع الفهرسة](#)
- [توسعة الأنواع](#)
- [الأنواع الحرفية](#)
- [استدلال الأنواع الحرفية](#)
- [strictNullChecks](#)
- [التعدادات](#)
 - [التعدادات العددية](#)
 - [التعدادات النصية](#)
 - [التعدادات الثابتة](#)
 - [الربط العكسي](#)
 - [التعدادات المحيطية](#)
 - [الأعضاء المحسوبة والثابتة](#)
- [التضييق](#)
 - [typeof حراس الأنواع باستخدام](#)
 - [التضييق بحسب الصدقية](#)
 - [التضييق بحسب المساواة](#)
 - [In التضييق باستخدام المعامل](#)
 - [instanceof التضييق باستخدام](#)
- [الإسنادات](#)
- [تحليل تدفق التحكم](#)
- [مسنندات الأنواع](#)
- [الاتحادات المميزة](#)
- [never النوع](#)
- [التحقق من الشمول](#)

- أنواع الكائنات
- (مجهول الاسم) Tuple نوع
- (المسمّى) الموسوم Tuple نوع
- ثابت الطول Tuple
- نوع الاتحاد
- أنواع التقاطع
- فهرسة الأنواع
- النوع من القيمة
- النوع من القيمة المرجعة للدالة
- النوع من الوحدة
- الأنواع المعيّنة
- معدّلات الأنواع المعيّنة
- الأنواع الشرطية
- الأنواع الشرطية التوزيعية
- في الأنواع الشرطية infer استدلال النوع باستخدام
- الأنواع الشرطية المعرّفة مسبقًا
- أنواع اتحادات القوالب
- النوع Any
- النوع Unknown
- النوع Void
- نوع Never
- الواجهة والنوع
 - الصياغة الشائعة
 - الأنواع الأساسية
 - الكائنات والواجهات
 - أنواع الاتحاد والتقاطع
- الأنواع الأولية المضمّنة
- المضمّنة الشائعة JS كائنات
- التحميلات الزائدة
- الدمج والتوسيع
- الاختلافات بين النوع والواجهة
- الصف

- [الصياغة الشائعة للصنف](#)
- [المُنشئ](#)
- [المُنشئات الخاصة والمحمية](#)
- [محددات الوصول](#)
- [الجليب والضبط](#)
- [الموصلات التلقائية في الأصناف](#)
- [this](#)
- [خصائص المعاملات](#)
- [الأصناف المجردة](#)
- [باستخدام الأنواع العامة](#)
- [المزخرفات](#)
 - [مزخرفات الأصناف](#)
 - [مزخرف الخاصة](#)
 - [مزخرف الأسلوب](#)
 - [مزخرفات دوال الجليب والضبط](#)
 - [البيانات الوصفية للمزخرفات](#)
- [الوراثة](#)
- [الأعضاء الساكنة](#)
- [تهيئة الخصائص](#)
- [التحميل الزائد للأساليب](#)
- [الأنواع العامة](#)
 - [النوع العام](#)
 - [الأصناف العامة](#)
 - [قيود الأنواع العامة](#)
 - [التضييق السياقي للأنواع العامة](#)
- [الأنواع البنيوية المحوطة](#)
- [مساحات الأسماء](#)
- [الرموز](#)
- [توجيهات الشروط المائلة الثلاث](#)
- [معالجة الأنواع](#)
 - [إنشاء أنواع من أنواع أخرى](#)
 - [أنواع الوصول المفهرس](#)

- أنواع الأدوات المساعدة
 - Awaited<T>
 - Partial<T>
 - Required<T>
 - Readonly<T>
 - Record<K, T>
 - Pick<T, K>
 - Omit<T, K>
 - Exclude<T, U>
 - Extract<T, U>
 - NonNullable<T>
 - Parameters<T>
 - ConstructorParameters<T>
 - ReturnType<T>
 - InstanceType<T>
 - ThisParameterType<T>
 - OmitThisParameter<T>
 - ThisType<T>
 - Uppercase<T>
 - Lowercase<T>
 - Capitalize<T>
 - Uncapitalize<T>
 - NoInfer<T>

- مواضيع أخرى
 - الأخطاء ومعالجة الاستثناءات
 - أصناف المزج
 - مميزات اللغة غير المتزامنة
 - المكثرات والمولدات
 - TsDocs و JSDoc مرجع
 - @types
 - JSX
 - ES6 وحدات

- [ES7 عامل الأس في](#)
- [for-await-of عبارة](#)
- [new.target الخاصية الوصفية](#)
- [تعبيرات الاستيراد الديناميكي](#)
- [“tsc –watch”](#)
- [عامل التوكيد بعدم انعدام القيمة](#)
- [التصريحات ذات القيم الافتراضية](#)
- [التسلسل الاختياري](#)
- [عامل الدمج المنعدم](#)
- [أنواع قوالب النصوص الحرفية](#)
- [التحميل الزائد للدوال](#)
- [الأنواع العودية](#)
- [الأنواع الشرطية العودية](#)
- [Node في ECMAScript دعم وحدات](#)
- [دوال التوكيد](#)
- [أنواع الصفوف متغيرة الطول](#)
- [الأنواع المغلفة](#)
- [TypeScript التغير والتغير العكسي في](#)
 - [تعليقات التغير الاختيارية لمعاملات الأنواع](#)
- [توابع الفهرس ذات أنماط السلاسل القالبية](#)
- [satisfies المعامل](#)
- [عمليات الاستيراد والتصدير الخاصة بالأنواع فقط](#)
- [والإدارة الصريحة للموارد using تصريح](#)
 - [await using تصريح](#)
- [سمات الاستيراد](#)
- [التحقق من صياغة التعبيرات النمطية](#)
- [import defer](#)

مقدمة

الموجز! يزودك هذا الدليل بالمعرفة TypeScript مرحبًا بك في كتاب بفعالية. TypeScript الأساسية والمهارات العملية اللازمة لتطوير تطبيقات اكتشف المفاهيم والتقنيات الأساسية لكتابة شيفرة نظيفة ومتينة. سواء أكنت مبتدئًا أم مطورًا متمرسًا، فإن هذا الكتاب يشكّل دليلًا شاملاً ومرجعًا مفيدًا في مشروعاتك TypeScript في آن واحد للاستفادة من قوة

TypeScript 7.0. يغطي هذا الكتاب

عن المؤلف

هو مهندس برمجيات رئيسي متمرس وشغوف بكتابة Simone Poggiali شيفرة برمجية احترافية منذ تسعينيات القرن الماضي. وقد ساهم طوال مسيرته المهنية الدولية في مشروعات عديدة لمجموعة واسعة من العملاء، بدءًا من الشركات الناشئة وصولًا إلى المؤسسات الكبرى. واستفادت شركات من Snowplow وLeroy Merlin وO2 وSiemens وHelloFresh بارزة مثل خبرته وتفانيه.

عبر المنصات التالية Simone Poggiali يمكنك التواصل مع:

- LinkedIn: <https://www.linkedin.com/in/simone-poggiali>
- GitHub: <https://github.com/gibbok>
- X.com: https://x.com/gibbok_coding
- البريد الإلكتروني: gibbok.coding@gmail.com

القائمة الكاملة للمساهمين: <https://github.com/gibbok/typescript-book/graphs/contributors>

TypeScript مقدمة إلى

TypeScript ما

صمّمها JavaScript لغة برمجة ذات تنميط قوي مبنية على TypeScript حاليًا تطويرها Microsoft في الأصل عام 2012، وتولى Anders Hejlsberg

وصيانتها بوصفها مشروعًا مفتوح المصدر.

ويمكن تنفيذها في أي بيئة تشغيل JavaScript إلى TypeScript تُصرَّف (على خادم Node.js مثل المتصفح أو JavaScript).

تدعم أنماط برمجة متعددة، مثل البرمجة الوظيفية، والعامة، والأمرية، قبل التنفيذ JavaScript وكائنية التوجّه، وهي لغة مصرّفة (محوّلة) تُحوّل إلى

لماذا TypeScript؟

لغة ذات تنميط قوي تساعد على منع أخطاء البرمجة الشائعة TypeScript وتجنّب أنواع معيّنة من أخطاء وقت التشغيل قبل تنفيذ البرنامج

تتيح اللغة ذات التنميط القوي للمطوّر تحديد قيود وسلوكيات مختلفة للبرنامج في تعريفات أنواع البيانات، ما يسهّل التحقق من صحة البرمجيات ومنع العيوب. وتبرز قيمة ذلك بوجه خاص في التطبيقات واسعة النطاق.

TypeScript من فوائد:

- التنميط الساكن، مع تنميط قوي اختياري
- استدلال النوع
- ES6 و ES7 الوصول إلى ميزات
- التوافق عبر المنصات والمتصفحات
- IntelliSense دعم الأدوات باستخدام

TypeScript و JavaScript

JavaScript بينما تُكتب ملفات .tsx أو .ts في ملفات TypeScript تُكتب .jsx أو .js في ملفات

أن تحتوي على امتداد لصياغة .jsx أو .tsx. يمكن للملفات ذات الامتداد لتطوير واجهات المستخدم React والمستخدم في JSX يُسمّى JavaScript

JavaScript من حيث البنية، مجموعة شاملة من TypeScript تُعد هي شيفرة JavaScript مع إضافة الأنواع. كل شيفرة (ECMAScript 2015) صالحة، لكن العكس ليس صحيحًا دائمًا TypeScript.

مثل، js. بالامتداد JavaScript على سبيل المثال، لنتناول دالة في ملف الآتي:

```
const sum = (a, b) => a + b;
```

بتغيير امتداد الملف إلى TypeScript يمكن تحويل الدالة واستخدامها في إلى الدالة نفسها، TypeScript لكن إذا أُضيفت تعليقات توضيحية لأنواع .ts. من دون تصريح. ستنجح JavaScript فلن يمكن تنفيذها في أي بيئة تشغيل التالية خطأ في البنية إذا لم تُصرّف TypeScript شيفرة

```
const sum = (a: number, b: number): number => a + b;
```

لاكتشاف أخطاء وقت التشغيل المحتملة في وقت TypeScript صُممت التصريف، وذلك عبر السماح للمطوّرين بالتعبير عن مقاصدهم باستخدام اكتشاف TypeScript التعليقات التوضيحية للأنواع. وإضافةً إلى ذلك، تستطيع بعض المشكلات حتى عند عدم تقديم تعليقات توضيحية صريحة للأنواع، بفضل استدلال النوع. على سبيل المثال، لا تحدد مقتطفة الشيفرة التالية أي أنواع TypeScript:

```
const items = [{ x: 1 }, { x: 2 }];  
const result = items.filter(item => item.y);
```

خطأً وتبلغ عنه TypeScript في هذه الحالة، تكتشف:

Property 'y' does not exist on type '{ x: number; }'.

في JavaScript إلى حد كبير بسلوك TypeScript يتأثر نظام الأنواع في وقت التشغيل. فمثلاً، تجري نمذجة معامل الجمع (+)، الذي يمكنه في إجراء ربط النصوص أو الجمع العددي، بالطريقة نفسها في JavaScript TypeScript:

```
const result = '1' + 1; // Result is of type string
```

قرارًا متعمدًا باعتبار الاستخدامات TypeScript اتخذ الفريق المسؤول عن أخطاءً. على سبيل المثال، لنتناول شيفرة JavaScript غير المعتادة في الصالحة التالية JavaScript:

```
const result = 1 + true; // In JavaScript, the result is equal to 1
```

تُصدر خطأ TypeScript لكن:

Operator '+' cannot be applied to types 'number' and 'boolean'.

تفرض توافق الأنواع بصرامة، وتتعرف في TypeScript يحدث هذا الخطأ لأن هذه الحالة على عملية غير صالحة بين عدد وقيمة منطقية.

TypeScript توليد الشيفرة في

مسؤوليتين رئيسيتين: التحقق من أخطاء الأنواع TypeScript يتحمل مصرّف وهاتان العمليتان مستقلتان إحداهما عن الأخرى. JavaScript والتصريف إلى لأنها تُمحي، JavaScript لا تؤثر الأنواع في تنفيذ الشيفرة داخل بيئة تشغيل إخراج شيفرة TypeScript بالكامل أثناء التصريف. ولا يزال بإمكان حتى في وجود أخطاء في الأنواع. فيما يلي مثال على شيفرة JavaScript تحتوي على خطأ في النوع TypeScript:

```
const add = (a: number, b: number): number => a + b;  
const result = add('x', 'y'); // Argument of type 'string' is not assignable to parameter of type 'number'
```

قابلة للتنفيذ JavaScript ومع ذلك، لا يزال بإمكانها إنتاج مخرجات:

```
'use strict';  
const add = (a, b) => a + b;  
const result = add('x', 'y'); // xy
```


في وقت التشغيل. على سبيل المثال TypeScript لا يمكن التحقق من أنواع

```
interface Animal {
  name: string;
}
interface Dog extends Animal {
  bark: () => void;
}
interface Cat extends Animal {
  meow: () => void;
}
const makeNoise = (animal: Animal) => {
  if (animal instanceof Dog) {
    // 'Dog' only refers to a type, but is being used as a \
    // ...
  }
};
```

نظرًا إلى أن الأنواع تُمحي بعد التصريف، فلا توجد طريقة لتشغيل هذه وللتعرّف على الأنواع في وقت التشغيل، نحتاج JavaScript الشيفرة في عدة خيارات، ومن الخيارات TypeScript إلى استخدام آلية أخرى. توفر الشائعة «الاتحاد الموسوم». على سبيل المثال:

```
interface Dog {
  kind: 'dog'; // Tagged union
  bark: () => void;
}
interface Cat {
  kind: 'cat'; // Tagged union
  meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
  if (animal.kind === 'dog') {
    animal.bark();
  } else {
```

```

        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);

```

هي قيمة يمكن استخدامها في وقت التشغيل للتمييز بين “kind” الخاصة JavaScript الكائنات في.

ومن الممكن أيضًا أن يكون لقيمة في وقت التشغيل نوع مختلف عن النوع المصرّح به في تصريح النوع. يحدث ذلك مثلًا إذا أساء المطوّر تفسير نوع وأضاف إليه تعليقًا توضيحيًا غير صحيح API واجهة.

لذا يمكن استخدام الكلمة JavaScript، مجموعة شاملة من TypeScript بوصفها نوعًا وقيمةً في وقت التشغيل “class” المفتاحية.

```

class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}

```

```

    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);

```

ويمكن استخدام "prototype" خاصية "class" يمتلك JavaScript، في لمنشئ ما prototype لاختبار ما إذا كانت خاصية "instanceof" المعامل لكائن prototype تظهر في أي موضع ضمن سلسلة.

أي تأثير في أداء وقت التشغيل، إذ سُمحى جميع TypeScript ليس لـ. تضيف بعض الحمل الزائد إلى وقت البناء TypeScript الأنواع. لكن

(الحدثة الآن) التحويل إلى إصدار أقدم JavaScript

تصريف الشيفرة إلى أي إصدار منشور من TypeScript يمكن لـ TypeScript (1999). وهذا يعني أن ECMAScript 3 منذ JavaScript إلى JavaScript تستطيع تحويل الشيفرة التي تستخدم أحدث ميزات ويتيح ذلك Downleveling. إصدارات أقدم، وهي عملية تُعرف باسم الحديثة مع الحفاظ على أقصى قدر من التوافق مع JavaScript استخدام. بيئات التشغيل القديمة.

قد تولّد JavaScript، من المهم ملاحظة أنه أثناء التحويل إلى إصدار أقدم من شيفرة يمكن أن تفرض حملًا إضافيًا على الأداء مقارنةً TypeScript بالتنفيذات الأصلية.

الحديثة التي يمكن استخدامها في JavaScript فيما يلي بعض ميزات TypeScript:

- بأسلوب “define” بدلاً من استدعاءات رد النداء ECMAScript وحدات CommonJS. في “require” أو عبارات AMD.
- prototypes الأصناف بدلاً من.
- “var” بدلاً من “const” أو “let” التصريح عن المتغيرات باستخدام.
- التقليدية “for” بدلاً من حلقة “forEach.” أو “for-of” حلقة.
- الدوال السهمية بدلاً من تعبيرات الدوال.
- إسناد التفكيك.
- الأسماء المختصرة للخصائص/الأساليب وأسماء الخصائص المحسوبة.
- المعاملات الافتراضية للدوال.

الحديثة هذه، يمكن للمطورين JavaScript من خلال الاستفادة من ميزات TypeScript كتابة شيفرة أكثر تعبيرًا وإيجازًا في.

TypeScript بدء استخدام

التثبيت

لكنه لا يتضمن TypeScript دعمًا ممتازًا للغة Visual Studio Code يوفر يمكنك استخدام مدير، TypeScript لتثبيت مصرف TypeScript مصرف مثل yarn أو npm حزم مثل:

```
npm install typescript --save-dev
```

أو

```
yarn add typescript --dev
```

تأكد من إيداع ملف القفل الناتج لضمان استخدام كل عضو في الفريق نفسه TypeScript لإصدار.

يمكنك استخدام الأوامر التالية، TypeScript لتشغيل مصرف

```
npx tsc
```

أو

```
yarn tsc
```

على مستوى المشروع بدلاً من تثبيتها عمومياً، لأن TypeScript يُوصى بتثبيت ذلك يوفر عملية بناء يمكن التنبؤ بها بدرجة أكبر. لكن للاستخدامات العابرة، يمكنك استخدام الأمر التالي:

```
npx tsc
```

:أو تثبيتها عمومياً

```
npm install -g typescript
```

فيمكنك الحصول على Microsoft Visual Studio، إذا كنت تستخدم NuGet. لمشروعات MSBuild. بوصفها حزمة في TypeScript NuGet التالي في وحدة تحكم مدير حزم:

```
Install-Package Microsoft.TypeScript.MSBuild
```

بوصفه مصرّف "tsc": يُثبت ملفان تنفيذيان، TypeScript أثناء تثبيت المستقل. يحتوي TypeScript بوصفه خادم "tsserver" و TypeScript الخادم المستقل على المصرّف وخدمات اللغة التي يمكن للمحرّرات وبيئات التطوير المتكاملة استخدامها لتوفير الإكمال الذكي للشفيرة.

Babel مثل TypeScript، إضافةً إلى ذلك، تتوفر عدة محوّلات متوافقة مع ويمكن استخدام هذه المحوّلات لتحويل شيفرة SWC عبر مكوّن إضافي (أو) إلى لغات أو إصدارات مستهدفة أخرى TypeScript.

بوصفها تنفيذاً أصلياً للمصرّف وخدمة Go بلغة TypeScript 7.0 أُعيدت كتابة اللغة. وهي تستخدم تعدد خيوط التنفيذ ذات الذاكرة المشتركة وتحسينات أخرى لجعل عمليات البناء الكاملة وميزات المحرّر أسرع، ما يقلل زمن الحصول على الملاحظات أثناء التطوير.

يمكن تشغيل التحقق TypeScript 7.0 يمكن ضبط بعض ميزات الأداء في ؛ وقد يؤدي --checkers من الأنواع في عمليات عاملة متوازية باستخدام المزيد من العمليات العاملة إلى تسريع المشروعات الكبيرة، لكنه يستهلك المعاد بناؤه مراقبة الملفات عبر المنصات. --watch ذاكرة أكبر. يحسن وضع للمصرف (حتى يوليو API حتى الآن واجهة TypeScript 7.0 لا تتضمن في API 2026)، ولذلك يمكن للأدوات التي لا تزال تحتاج إلى واجهة باستخدام TypeScript 7.0 تشغيلها بالتوازي مع TypeScript 6.0 npm. أو الأسماء البديلة في @typescript/typescript6

التهيئة

أو عبر tsc باستخدام خيارات واجهة سطر أوامر TypeScript يمكن تهيئة ويوضع في جذر tsconfig.json استخدام ملف تهيئة مخصص يُسمى المشروع.

مجهّز مسبقًا بالإعدادات الموصى بها، يمكنك tsconfig.json لإنشاء ملف استخدام الأمر التالي:

```
tsc --init
```

الشفرة باستخدام التهيئة TypeScript محليًا، ستصرف tsc عند تنفيذ الأمر tsconfig.json المحددة في أقرب ملف

فيما يلي بعض أمثلة أوامر واجهة سطر الأوامر التي تعمل بالإعدادات الافتراضية:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

TypeScript ملف تهيئة

وعادةً ما (tsc) TypeScript لتهيئة مصرّف tsconfig.json يُستخدم ملف package.json. يُضاف إلى جذر المشروع إلى جانب ملف

ملاحظات:

- json التعليقات حتى وإن كان بتنسيق tsconfig.json يقبل.
- يُنصح باستخدام ملف التهيئة هذا بدلاً من خيارات سطر الأوامر.

يمكنك العثور في الرابط التالي على الوثائق الكاملة والمخطط الخاص به:

<https://www.typescriptlang.org/tsconfig>

<https://www.typescriptlang.org/tsconfig/>

تمثل القائمة التالية مجموعة من إعدادات التهيئة الشائعة والمفيدة:

target

الذي ينبغي أن تُخرج ECMAScript لتحديد إصدار "target" تُستخدم الخاصية خيارًا جيدًا للمتصفحات ES6 وتُعد TypeScript أو تُصرّف إليه شيفرة ولم يعد مدعومًا في TypeScript 6.0، في ES5 الحديثة. ملاحظة: أهمل دعم TypeScript 7.0.

lib

لتحديد ملفات المكتبة التي يجب تضمينها في وقت "lib" تُستخدم الخاصية للميزات المحددة في API تلقائيًا واجهات TypeScript التصريف. تضمّن لكن يمكن استبعاد مكتبات بعينها أو اختيارها لتلبية "target" الخاصية احتياجات محددة. فإذا كنت تعمل على مشروع خادم مثلاً، يمكنك استبعاد التي لا تفيد إلا في بيئة المتصفح "DOM" مكتبة

strict

أمان الأنواع بتمكين عمليات تحقق أقوى. وهو مفعّل "strict" يحسّن الخيار ؛ وفي غير ذلك، ينبغي ضبطه صراحةً على TypeScript 6.0 افتراضياً بدءاً من ما يلي TypeScript لـ "strict" يتيح تمكين. tsconfig.json في ملف true

- لكل ملف مصدر "use strict" إخراج الشيفرة باستخدام
- في عملية التحقق من الأنواع "undefined" و "null" مراعاة
- عند عدم وجود تعليقات توضيحية للأنواع "any" تعطيل استخدام النوع
- الذي كان سيعني خلاف ذلك "this" إصدار خطأ عند استخدام التعبير "any".

module

نظام الوحدات المدعوم للبرنامج المصنّف. وفي "module" تحدد الخاصية وقت التشغيل، يُستخدم محمّل وحدات لتحديد التبعيات وتنفيذها استناداً إلى نظام الوحدات المحدد.

في CommonJS هما JavaScript أكثر محمّلات الوحدات استخداماً في في تطبيقات AMD لوحات RequireJS لتطبيقات جانب الخادم و Node.js إخراج شيفرة لأنظمة TypeScript الويب القائمة على المتصفح. ويمكن لـ ES2020 و ES2015/ES6 و ESNext و System و UMD وحدات مختلفة، منها وينبغي اختيار نظام الوحدات استناداً إلى البيئة المستهدفة وآلية تحميل الوحدات المتاحة فيها.

(AMD و UMD و SystemJS) ملاحظة: أهمل دعم أنظمة الوحدات القديمة TypeScript 7.0 ولم تعد مدعومة في TypeScript 6.0 في

moduleResolution

استراتيجية تحليل الوحدات. استخدم "moduleResolution" تحدد الخاصية الحديثة. ولا تُستخدم TypeScript لشيفرة "bundler" أو "nodenext" قبل TypeScript إلا مع الإصدارات القديمة من "classic" الاستراتيجية (1.6).

esModuleInterop

بعمليات الاستيراد الافتراضية من "esModuleInterop" تسمح الخاصية ؛ وتوفّر "default" التي لم تُصدّر باستخدام الخاصية CommonJS وحدات الناتجة. وبعد JavaScript هذه الخاصية طبقة توافق لضمان التوافق في `import MyLibrary from "my-library"` يمكن هذا الخيار، يمكننا استخدام `import * as MyLibrary from "my-library"` بدلاً من.

في الأصل خيارًا صريحًا لتجنب التغييرات "esModuleInterop" كانت الكاسرة، لكنها أصبحت منذ وقت طويل الإعداد الافتراضي الموصى به. وقد يؤدي تعطيلها إلى مشكلات دقيقة في وقت التشغيل عند استخدام يكون سلوك TypeScript 6.0 ملاحظة: بدءًا من ESM. مع CommonJS التشغيل البيئي الأكثر أمانًا هذا مفعّلًا دائمًا.

أُهملت بعض خيارات التهيئة وأشكال البنية القديمة أو TypeScript 6.0 في أصبحت TypeScript 7.0 مرّت بمرحلة انتقالية عبر السلوك القديم. وفي أخطاءً مانعة أو ذات سلوك لا يُجري أي عملية.

الخيارات المهمة التي تحوّلت إلى أخطاء مانعة ذات سلوك لا يُجري أي عملية هي:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- `allowSyntheticDefaultImports` أو `esModuleInterop` تعطيل
- `alwaysStrict` تعطيل
- في تصريحات مساحة الاسم `module` الكلمة المفتاحية
- في عمليات الاستيراد `asserts`
- `skipDefaultLibCheck` ضمن `/// <reference no-default-lib />`

- مسارات ملفات واجهة سطر الأوامر التي تحتوي محليًا على `--ignoreConfig` ما لم يُستخدم `tsconfig.json`

jsx

ReactJS المستخدمة في `tsx`. فقط على ملفات `jsx` تنطبق الخاصية ومن الخيارات الشائعة. JavaScript إلى JSX وتتحكم في كيفية تصريف بُنى من دون تغيير، بحيث JSX مع إبقاء `jsx`. الذي يصرّف إلى ملف `“preserve”` لإجراء المزيد من التحويلات Babel يمكن تمريره إلى أدوات مختلفة مثل.

skipLibCheck

من التحقق من أنواع حزم TypeScript `“skipLibCheck”` تمنع الخاصية الجهات الخارجية المستوردة بأكملها. وتقلل هذه الخاصية وقت تصريف تتحقق من شيفرتك في مقابل تعريفات TypeScript المشروع. وستظل الأنواع التي توقّرها هذه الحزم.

files

للمصرّف إلى قائمة الملفات التي يجب تضمينها دائمًا `“files”` تشير الخاصية في البرنامج.

include

للمصرّف إلى قائمة الملفات التي نرغب في `“include”` تشير الخاصية مثل `“*”` لأي دليل، `glob` تضمينها. وتسمح هذه الخاصية بأنماط شبيهة بأنماط فرعي، و `“”` لأي اسم ملف، و `“?”` للمحارف الاختيارية.

exclude

للمصرّف إلى قائمة الملفات التي ينبغي عدم `“exclude”` تشير الخاصية أو `“node_modules”` تضمينها في التصريف. ويمكن أن تشمل ملفات مثل

بالتعليقات tsconfig.json ملفات الاختبار. ملاحظة: يسمح

importHelpers

شيفرة مساعدة عند توليد الشيفرة لبعض ميزات TypeScript تستخدم المتقدمة أو الميزات التي تُحوَّل إلى إصدارات أقدم. افتراضياً، JavaScript تتكرر هذه الدوال المساعدة في الملفات التي تستخدمها. أما الخيار بدلاً من tslib فيستورد هذه الدوال المساعدة من الوحدة importHelpers أكثر كفاءة JavaScript ذلك، ما يجعل مخرجات

TypeScript نصائح للانتقال إلى

بالنسبة إلى المشاريع الكبيرة، يُنصح باتباع انتقال تدريجي تتعايش فيه في البداية. ولا يمكن الانتقال إلى JavaScript و TypeScript شيفرات دفعة واحدة إلا في المشاريع الصغيرة TypeScript.

ضمن سلسلة TypeScript تتمثل الخطوة الأولى في هذا الانتقال في إدخال الذي، "allowJs" عملية البناء. ويمكن فعل ذلك باستخدام خيار المصرّف الموجودة. وبما أن JavaScript بالتعايش مع ملفات tsx و ts. يسمح لملفات للمتغير عندما يتعذر عليه استدلال النوع "any" سيلجأ إلى النوع TypeScript في خيارات "noImplicitAny" فيُنصح بتعطيل JavaScript من ملفات المصرّف في بداية الانتقال.

تعمل إلى جانب JavaScript تتمثل الخطوة الثانية في التأكد من أن اختبارات حتى تتمكن من تشغيل الاختبارات أثناء تحويل كل TypeScript ملفات الذي يتيح لك ts-jest، ففكر في استخدام Jest وحدة. وإذا كنت تستخدم Jest باستخدام TypeScript اختبار مشاريع.

تتمثل الخطوة الثالثة في تضمين تصريحات الأنواع الخاصة بمكتبات الجهات الخارجية في مشروعك. ويمكن العثور على هذه التصريحات إما مضمّنة مع ويمكنك البحث عنها باستخدام DefinitelyTyped. المكتبات أو على وتثبيتها باستخدام <https://www.typescriptlang.org/dt/search>

```
npm install --save-dev @types/package-name
```

أو

```
yarn add --dev @types/package-name
```

تتمثل الخطوة الرابعة في الانتقال وحدةً تلو الأخرى باتباع نهج من الأسفل إلى الأعلى، مع تتبّع مخطط التبعيات بدءًا من الأوراق. والفكرة هي البدء بتحويل الوحدات التي لا تعتمد على وحدات أخرى. ولعرض مخططات “madge” التبعيات مرئيًا، يمكنك استخدام أداة

من الوحدات المرشحة جيدًا لهذه التحويلات الأولية دوال الأدوات المساعدة الخارجية أو المواصفات. ومن الممكن API والشيفرة المرتبطة بواجهات أو مخططات Swagger تلقائيًا من عقود TypeScript توليد تعريفات أنواع لتضمينها في مشروعك JSON أو GraphQL.

عندما لا تتوفر مواصفات أو مخططات رسمية، يمكنك توليد الأنواع من الذي يعيده الخادم. ومع ذلك، يُنصح بتوليد الأنواع JSON البيانات الأولية، مثل من المواصفات بدلًا من البيانات لتجنب إغفال الحالات الحديثة.

أثناء الانتقال، امتنع عن إعادة هيكلة الشيفرة وركّز فقط على إضافة الأنواع إلى وحداتك.

ما سيفرض معرفة “noImplicitAny” تتمثل الخطوة الخامسة في تفعيل TypeScript جميع الأنواع وتعريفها، ويوفر تجربة أفضل لاستخدام مشروعك.

الذي يفعل التحقق من @ts-check أثناء الانتقال، يمكنك استخدام التوجيه يوفر هذا التوجيه إصدارًا متساهلاً JavaScript. داخل ملف TypeScript أنواع من التحقق من الأنواع، ويمكن استخدامه في البداية لتحديد المشكلات في في ملف، ستحاول @ts-check وعندما يُضمّن JavaScript. ملفات JSDoc. ومع استدلال التعريفات باستخدام تعليقات بأسلوب TypeScript التوضيحية في مرحلة مبكرة جدًا من JSDoc ذلك، فركّز في استخدام تعليقات الانتقال فقط.

في ملف `noEmitOnError` فُكّر في إبقاء القيمة الافتراضية للخيار `tsconfig.json` على `false`. JavaScript سيسمح لك ذلك بإخراج شيفرة المصدرية حتى عند الإبلاغ عن أخطاء.

استكشاف نظام الأنواع

TypeScript خدمة لغة

مميزات متعددة، `tsserver` المعروفة أيضًا باسم `TypeScript` تقدم خدمة لغة مثل الإبلاغ عن الأخطاء، والتشخيصات، والتصريف عند الحفظ، وإعادة التسمية، والانتقال إلى التعريف، وقوائم الإكمال، والمساعدة الخاصة لتوفير (IDEs) بالتوافق، وغيرها. وتستخدمها أساسًا بيئات التطوير المتكاملة `Visual Studio Code` وهي تتكامل بسلاسة مع `IntelliSense`. `Conquer of Completion (Coc)` وتستخدمها أدوات مثل

مخصصة وإنشاء إضافاتهم الخاصة API يمكن للمطورين الاستفادة من واجهة وقد يكون ذلك `TypeScript` والمخصصة لخدمة اللغة لتحسين تجربة تحرير مفيدًا على نحو خاص لتنفيذ ميزات تدقيق خاصة أو لتمكين الإكمال التلقائي للغة قوالب مخصصة.

“`typescript-styled-plugin`” من الأمثلة الواقعية على الإضافات المخصصة `IntelliSense` التي توفر الإبلاغ عن أخطاء بناء الجملة ودعم، في المكونات المنسقة CSS لخصائص.

لمزيد من المعلومات وأدلة البدء السريع، يمكنك الرجوع إلى ويكي `GitHub` الرسمية على `TypeScript`:

<https://github.com/microsoft/TypeScript/wiki/>

التميط البنيوي

على نظام أنواع بنيوي. وهذا يعني أن توافق الأنواع `TypeScript` تعتمد وتكافؤها يتحددان من خلال البنية أو التعريف الفعلي للنوع، وليس اسمه أو `C` أو `C#` موضع إعلانه، كما في أنظمة الأنواع الاسمية مثل

استنادًا إلى طريقة عمل نظام TypeScript صُمم نظام الأنواع البنيوي في أثناء وقت JavaScript الترميز الديناميكي القائم على التوافق البنيوي في التشغيل.

”Y” و”X” صالحة. وكما تلاحظ، يحتوي TypeScript المثال التالي هو شيفرة رغم اختلاف اسمي التصريح عنهما. تتحدد الأنواع من ”a” على العضو نفسه خلال بنيتها، وفي هذه الحالة، بما أن البنيتين متماثلتان، فهما متوافقتان وصالحتان.

```
type X = {  
  a: string;  
};  
type Y = {  
  a: string;  
};  
const x: X = { a: 'a' };  
const y: Y = x; // Valid
```

TypeScript قواعد المقارنة الأساسية في

تعاودية وتُنفذ على الأنواع المتداخلة في أي TypeScript عملية المقارنة في مستوى.

يحتوي على الأقل على أعضاء ”Y” إذا كان ”Y” متوافقًا مع ”X” يكون النوع نفسه ”X”.

```
type X = {  
  a: string;  
};  
const y = { a: 'A', b: 'B' }; // Valid, as it has at least the s  
const r: X = y;
```

تُقارن معاملات الدوال حسب أنواعها، وليس حسب أسمائها

```

type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Valid
x = y; // Valid

```

يجب أن تكون أنواع إرجاع الدوال متماثلة:

```

type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalid
x = y; // Invalid

```

يجب أن يكون نوع إرجاع الدالة المصدر نوعًا فرعيًا من نوع إرجاع الدالة الهدف:

```

let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Valid
y = x; // Invalid member b is missing

```

مثل JavaScript، يُسمح بتجاهل معاملات الدوال، إذ إنها ممارسة شائعة في استخدام “Array.prototype.map()”:

```

[1, 2, 3].map((element, _index, _array) => element + 'x');

```

وبالتالي، تكون تصريحات الأنواع التالية صالحة تمامًا:

```

type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Missing b parameter
y = x; // Valid

```

تكون أي معاملات اختيارية إضافية في النوع المصدر صالحة:

```

type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; //Valid

```

تُعد أي معاملات اختيارية في النوع الهدف لا تقابلها معاملات في النوع المصدر صالحة، ولا تمثل خطأ:

```

type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; // Valid

```

يُعامل معامل البقية كسلسلة لا نهائية من المعاملات الاختيارية:

```

type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valid

```

تكون الدوال ذات الأحمال الزائدة صالحة إذا كان توقيع الحمل الزائد متوافقًا مع توقيع تنفيذها:

```

function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valid
x('a', 1); // Valid

```

```

function y(a: string): void; // Invalid, not compatible with imp
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}

```



```
y('a');  
y('a', 1);
```

تنجح مقارنة معاملات الدوال إذا كان من الممكن إسناد المعاملات المصدر (والهدف إلى الأنواع الفائقة أو الأنواع الفرعية) (التباين الثنائي).

```
// Supertype  
class X {  
  a: string;  
  constructor(value: string) {  
    this.a = value;  
  }  
}  
  
// Subtype  
class Y extends X {}  
  
// Subtype  
class Z extends X {}  
  
type GetA = (x: X) => string;  
const getA: GetA = x => x.a;  
  
// Bivariance does accept supertypes  
console.log(getA(new X('x'))); // Valid  
console.log(getA(new Y('Y'))); // Valid  
console.log(getA(new Z('z'))); // Valid
```

يمكن مقارنة التعدادات بالأرقام والعكس صحيح ويكون ذلك صالحًا، لكن مقارنة قيم تعداد من أنواع تعداد مختلفة غير صالحة.

```
enum X {  
  A,  
  B,  
}  
  
enum Y {  
  A,  
  B,  
  C,
```

```

}
const xa: number = X.A; // Valid
const ya: Y = 0; // Valid
X.A === Y.A; // Invalid

```

تخضع مثيلات الصنف لفحص توافق أعضائها الخاصة والمحمية:

```

class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Invalid

```

لا يأخذ فحص المقارنة في الحسبان اختلاف التسلسل الهرمي للوراثة، على سبيل المثال:

```

class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}

```

```

}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Valid
x === z; // Valid even if z is from a different inheritance hierarchy

```

تُقارن الأنواع العامة باستخدام بنيتها استنادًا إلى النوع الناتج بعد تطبيق معامل النوع العام، ولا يُقارن إلا الناتج النهائي بوصفه نوعًا غير عام.

```

interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalid as the type argument is used in the final signature

```

```

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Valid as the type argument is not used in the final signature

```

عندما لا يُحدّد وسيط النوع للأنواع العامة، تُعامل جميع الوسائط غير المحددة “any” على أنها من النوع:

```

type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Valid

```

تذكر:

```
let a: number = 1;
let b: number = 2;
a = b; // Valid, everything is assignable to itself

let c: any;
c = 1; // Valid, all types are assignable to any

let d: unknown;
d = 1; // Valid, all types are assignable to unknown

let e: unknown;
let e1: unknown = e; // Valid, unknown is only assignable to its
let e2: any = e; // Valid
let e3: number = e; // Invalid

let f: never;
f = 1; // Invalid, nothing is assignable to never

let g: void;
let g1: any;
g = 1; // Invalid, void is not assignable to or from anything else
g = g1; // Valid
```

”null” يُعامل، ”strictNullChecks” يرجى ملاحظة أنه عند تفعيل ؛ وإلا فيُعاملان على نحو مماثل لـ ”void” على نحو مماثل لـ ”undefined” و ”never”.

الأنواع بوصفها مجموعات

النوع هو مجموعة من القيم الممكنة. ويُشار إلى هذه، في TypeScript، في المجموعة أيضًا باسم مجال النوع. ويمكن النظر إلى كل قيمة من نوع ما على أنها عنصر في مجموعة. ويحدد النوع القيود التي يجب أن يستوفيها كل عنصر في TypeScript في المجموعة حتى يُعد عضوًا فيها. تتمثل المهمة الأساسية لـ

في فحص ما إذا كانت إحدى المجموعات مجموعة جزئية من أخرى والتحقق من ذلك.

أنواعًا مختلفة من المجموعات TypeScript تدعم:

ملاحظات	TypeScript	مصطلح المجموعة
على أي شيء باستثناء نفسه "never" لا يحتوي	never	المجموعة الخالية
	undefined /	مجموعة
	null /	أحادية
	literal type	العنصر
	boolean /	مجموعة
	union	منتهية
	string /	مجموعة
	number /	غير
	object	منتهية
وكل مجموعة مجموعة "any" كل عنصر عضو في النظير الآمن من "unknown" جزئية منه / يُعد "any" حيث الأنواع لـ	any /	المجموعة
	unknown	الشاملة

فيما يلي بعض الأمثلة:

مثال	مصطلح المجموعة	TypeScript
const x: never = 'x'; // Error: Type 'string' is not assignable to type 'never'	المجموعة (الخالية) $\emptyset$	never
type X = 'X'; type Y = 7;	مجموعة أحادية العنصر	Literal type

TypeScript	مصطلح المجموعة	مثال
Value assignable to T	$\in T$ القيمة (عضو في)	<pre>type XY = 'X' 'Y'; const x: XY = 'X';</pre>
T1 assignable to T2	$T1 \subseteq T2$ (مجموعة جزئية من)	<pre>type XY = 'X' 'Y'; const x: XY = 'X'; const j: XY = 'J'; // Type "J" is not assignable to type 'XY'.</pre>
T1 extends T2	$T1 \subseteq T2$ (مجموعة جزئية من)	<pre>type X = 'X' extends string ? true : false;</pre>
T1 T2	$T1 \cup T2$ (اتحاد)	<pre>type XY = 'X' 'Y'; type JK = 1 2;</pre>
T1 & T2	$T1 \cap T2$ (تقاطع)	<pre>type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: 'a', b: 'b' }</pre>
unknown	المجموعة الشاملة	<pre>const x: unknown = 1</pre>

(مجموعة أوسع) كلاهما، (T1 | T2)، ينشئ الاتحاد

```
type X = {
  a: string;
```

```
};
type Y = {
  b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valid
```

(مجموعة أضيق (المشترك فقط (T1 & T2)، ينشئ التقاطع

```
type X = {
  a: string;
};
type Y = {
  a: string;
  b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Invalid
const j: XY = { a: 'a', b: 'b' }; // Valid
```

بمعنى “مجموعة جزئية من” في هذا extends يمكن اعتبار الكلمة المفتاحية مع نوع عام، فإنها extends السياق. فهي تضع قيدًا لنوع ما. وعندما تُستخدم تقيّد معامل النوع العام بنوع أكثر تحديدًا.

هنا لا علاقة لها بوراثة الأصناف بمعنى البرمجة extends يرجى ملاحظة أن كائنية التوجه.

مع الأنواع البنيوية، وليس لديها تسلسل هرمي اسمي TypeScript تتعامل صارم. وفي الواقع، كما في المثال أدناه، يمكن أن يتداخل نوعان من دون أن تأخذ بنية الكائنات أو TypeScript يكون أي منهما نوعًا فرعيًا للآخر، لأن شكلها في الحساب.

```
interface X {
  a: string;
}
interface Y extends X {
  b: string;
```

```

}
interface Z extends Y {
  c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
  a: string;
}
interface Y1 {
  a: string;
  b: string;
}
interface Z1 {
  a: string;
  b: string;
  c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Valid

```

إسناد نوع: تصريحات الأنواع وتوكيدات الأنواع

TypeScript يمكن إسناد نوع بطرق مختلفة في

تصريح النوع

x. للتصريح بنوع المتغير (Type: أي) x في المثال التالي، نستخدم

```

type X = {
  a: string;
};

// Type declaration
const x: X = {
  a: 'a',
};

```


عن خطأ. على TypeScript إذا لم يكن المتغير بالتنسيق المحدد، فستبلغ سبيل المثال:

```
type X = {
  a: string;
};

const x: X = {
  a: 'a',
  b: 'b', // Error: Object literal may only specify known prop
};
```

توكيد النوع

يخبر هذا المصرّف بأن `as` يمكن إضافة توكيد باستخدام الكلمة المفتاحية `as`. لدى المطوّر معلومات أكثر عن النوع، ويمنع ظهور أي أخطاء قد تحدث.

على سبيل المثال:

```
type X = {
  a: string;
};

const x = {
  a: 'a',
  b: 'b',
} as X;
```

باستخدام الكلمة المفتاحية `X` من النوع `x` في المثال أعلاه، يُؤكّد أن الكائن `x` يتوافق مع النوع المحدد، رغم TypeScript ويُعلم ذلك مصرّف `as`. غير الموجودة في تعريف النوع `b` احتوائه على الخاصية الإضافية.

تكون توكيدات الأنواع مفيدة في الحالات التي يلزم فيها تحديد نوع أكثر على سبيل المثال. DOM. تخصيصًا، ولا سيما عند العمل مع

```
const myInput = document.getElementById('my_input') as HTMLInput
```

بأن TypeScript لإخبار `HTMLInputElement` as هنا، يُستخدم توكيد النوع `HTMLInputElement`. ينبغي أن تُعامل على أنها `getElementById` نتيجة يمكن أيضًا استخدام توكيدات الأنواع لإعادة تعيين المفاتيح، كما هو موضح في المثال أدناه باستخدام القوالب النصية:

```
type J<Type> = {
  [Property in keyof Type as `prefix_${string} &
    Property`]: () => Type[Property];
};
type X = {
  a: string;
  b: number;
};
type Y = J<X>;
```

نوعًا معيَّنًا مع قالب نصي لإعادة تعيين `J<Type>` في هذا المثال، يستخدم النوع في بداية كل `“prefix_”` وينشئ خصائص جديدة يُضاف إليها `Type`. مفاتيح مفتاح منها، وتكون قيمها المقابلة دوال تُرجع قيم الخصائص الأصلية.

لن تنفذ فحص الخصائص الزائدة عند TypeScript تجدر الإشارة إلى أن استخدام توكيد النوع. لذلك، يُفضَّل عمومًا استخدام تصريح النوع عندما تكون بنية الكائن معروفة مسبقًا.

التصريحات المحيطية

ويكون JavaScript، التصريحات المحيطية هي ملفات تصف أنواع شيفرة وعادةً ما تُستورد وتُستخدم لإضافة `.d.ts`. تنسيق اسم الملف الخاص بها الموجودة أو لإضافة أنواع إلى JavaScript تعليقات توضيحية إلى مكتبات الموجودة في مشروعك JS ملفات.

يمكن العثور على أنواع العديد من المكتبات الشائعة في:

<https://github.com/DefinitelyTyped/DefinitelyTyped/>

ويمكن تثبيتها باستخدام

```
npm install --save-dev @types/library-name
```

بالنسبة إلى التصريحات المحيطية التي عرّفتها، يمكنك الاستيراد باستخدام
”مرجع“ الشروط الثلاث:

```
/// <reference path="./library-types.d.ts" />
```

JavaScript يمكنك استخدام التصريحات المحيطية حتى داخل ملفات
// @ts-check. باستخدام

موجودة JavaScript تعريف الأنواع لشفرة declare تتيح الكلمة المفتاحية
من دون استيرادها، وتعمل بوصفها عنصرًا نائبًا للأنواع الواردة من ملف آخر
أو المتاحة عمومياً.

فحص الخصائص وفحص الخصائص الزائدة

على نظام أنواع بنيوي، لكن فحص الخصائص الزائدة هو TypeScript تعتمد
إحدى ميزاتها التي تتيح لها التحقق مما إذا كان الكائن يحتوي على الخصائص
المحددة في النوع بدقة.

يُجرى فحص الخصائص الزائدة عند إسناد الكائنات الحرفية إلى متغيرات، أو
عند تمريرها بوصفها وسائط إلى دالة، مثلاً.

```
type X = {  
  a: string;  
};  
  
const y = { a: 'a', b: 'b' };  
const x: X = y; // Valid because structural typing  
const w: X = { a: 'a', b: 'b' }; // Invalid because excess prop
```

الأنواع الضعيفة

يُعد النوع ضعيفاً عندما لا يحتوي إلا على مجموعة من الخصائص الاختيارية
بالكامل:

```
type X = {
  a?: string;
  b?: string;
};
```

إسناد أي شيء إلى نوع ضعيف خطأً عندما لا يوجد أي TypeScript تعدّ تداخل؛ فعلى سبيل المثال، يؤدي ما يلي إلى ظهور خطأً

```
type Options = {
  a?: string;
  b?: string;
};
```

```
const fn = (options: Options) => undefined;
```

```
fn({ c: 'c' }); // Invalid
```

رغم أن ذلك غير موصى به، يمكن عند الحاجة تجاوز هذا الفحص باستخدام توكيد النوع:

```
type Options = {
  a?: string;
  b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valid
```

إلى توقيع الفهرس في النوع الضعيف unknown أو بإضافة:

```
type Options = {
  [prop: string]: unknown;
  a?: string;
  b?: string;
};
```

```
const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valid
```

(الفحص الصارم للكائنات الحرفية (الحدث)

الفحص الصارم للكائنات الحرفية، الذي يُشار إليه أحيانًا باسم “الحدث”، هو تساعد على اكتشاف الخصائص الزائدة أو المكتوبة TypeScript ميزة في إملائيًا بصورة خاطئة، التي قد تمر دون ملاحظة في فحوص الأنواع البنيوية المعتادة.

حديثًا. وإذا أُسند الكائن TypeScript عند إنشاء كائن حرفي، يعدّه مصرّف خطأ إذا حدد TypeScript الحرفي إلى متغير أو مُرّر بوصفه معاملًا، فستطرح الكائن الحرفي خصائص غير موجودة في النوع الهدف.

لكن “الحدث” تختفي عند توسيع كائن حرفي أو استخدام توكيد النوع.

فيما يلي بعض الأمثلة للتوضيح:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness check: Invalid assignment
var y: Y;
y = { a: 'a', bx: 'bx' }; // Freshness check: Invalid assignment

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Widening: No errors, structurally type compatible

fn({ a: 'a', bx: 'b' }); // Freshness check: Invalid argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Widening: No Freshness check
```

استدلال الأنواع

استدلال الأنواع عند عدم توفير تعليق توضيحي في TypeScript يمكن لـ أثناء:

- تهيئة المتغير.
- تهيئة العضو.
- تعيين القيم الافتراضية للمعاملات.
- نوع إرجاع الدالة.

على سبيل المثال:

```
let x = 'x'; // The type inferred is string
```

القيمة أو التعبير، ويحدد نوعه استنادًا إلى TypeScript يحلّ مصرّف المعلومات المتاحة.

استدلالات أكثر تقدمًا

عن TypeScript عند استخدام عدة تعبيرات في استدلال الأنواع، تبحث “أفضل الأنواع المشتركة”. على سبيل المثال:

```
let x = [1, 'x', 1, null]; // The type inferred is: (string | nu
```

إذا لم يتمكن المصرّف من العثور على أفضل الأنواع المشتركة، فإنه يعيد نوع اتحاد. على سبيل المثال:

```
let x = [new RegExp('x'), new Date()]; // Type inferred is: (Reg
```

التميط السياقي “استنادًا إلى موضع المتغير” TypeScript تستخدم من النوع e لاستدلال الأنواع. في المثال التالي، يعرف المصرّف أن lib.d.ts، الذي المعرّف في الملف click بسبب نوع الحدث MouseEvent DOM الشائعة و JavaScript يحتوي على تصريحات محيطية لمختلف بُنى

```
window.addEventListener('click', function (e) {}); // The infer
```

توسيع النوع

نوعًا إلى متغير جرت TypeScript توسيع الأنواع هو العملية التي تُسند فيها تهيئته من دون تعليق توضيحي للنوع. ويسمح بالانتقال من الأنواع الضيقة إلى الأوسع، لا العكس. في المثال التالي:

```
let x = 'x'; // TypeScript infers as string, a wide type
let y: 'y' | 'x' = 'y'; // y types is a union of literal types
y = x; // Invalid Type 'string' is not assignable to type '"x"'
```

استنادًا إلى القيمة الوحيدة المقدمة x إلى string النوع TypeScript تُسند. وهذا مثال على التوسيع، (x) أثناء التهيئة.

“const” طرقًا للتحكم في عملية التوسيع، مثل استخدام TypeScript توفر.

Const

عند التصريح عن متغير إلى استدلال const يؤدي استخدام الكلمة المفتاحية TypeScript نوع أضيق في.

على سبيل المثال:

```
const x = 'x'; // TypeScript infers the type of x as 'x', a narrow type
let y: 'y' | 'x' = 'y';
y = x; // Valid: The type of x is inferred as 'x'
```

يُضَيَّق نوعه إلى القيمة الحرفية x، للتصريح عن المتغير const باستخدام من دون y قد ضُيِّق، فيمكن إسناده إلى المتغير x وبما أن نوع 'x' المحددة لا يمكن const أي خطأ. والسبب في إمكان استدلال النوع هو أن متغيرات إعادة الإسناد إليها، ولذلك يمكن تضيق نوعها إلى نوع حرفي محدد؛ وفي هذه الحالة، النوع الحرفي 'x'.

على معاملات الأنواع Const معدّل

على `const` يمكن تحديد السمة، TypeScript اعتبارًا من الإصدار 5.0 من معامل نوع عام. ويسمح ذلك باستدلال أدق نوع ممكن. لنرّ مثالًا من دون `const` استخدام:

```
function identity<T>(value: T) {  
    // No const here  
    return value;  
}  
const values = identity({ a: 'a', b: 'b' }); // Type inferred is
```

string. من النوع `b` و `a` كما ترى، يُستدل على أن الخاصيتين

`const` والآن، لنرّ الفرق مع إصدار

```
function identity<const T>(value: T) {  
    // Using const modifier on type parameters  
    return value;  
}  
const values = identity({ a: 'a', b: 'b' }); // Type inferred is
```

تُستنتجان بوصفهما قيمتين نصيتين `b` و `a` يمكننا الآن ملاحظة أن الخاصيتين `string` حرفيتين بدلًا من مجرد نوعي.

Const توكيد

تتيح لك هذه الميزة إعلان متغير بنوع حرفي أكثر دقة استنادًا إلى قيمة تهيئته، ما يشير للمصرّف إلى أن القيمة ينبغي أن تُعامل بوصفها قيمة حرفية غير قابلة للتغيير. وفيما يلي بعض الأمثلة:

على خاصية واحدة:

```
const v = {  
    x: 3 as const,  
};  
v.x = 3;
```


على كائن كامل:

```
const v = {  
  x: 1,  
  y: 2,  
} as const;
```

tuple: يمكن أن يكون هذا مفيدًا بصورة خاصة عند تعريف نوع

```
const x = [1, 2, 3]; // number[]  
const y = [1, 2, 3] as const; // Tuple of readonly [1, 2, 3]
```

التعليق التوضيحي الصريح للنوع

من النوع x يمكننا أن نكون محددين ونمرر نوعًا. في المثال التالي، الخاصية number:

```
const v = {  
  x: 1, // Inferred type: number (widening)  
};  
v.x = 3; // Valid
```

يمكننا جعل التعليق التوضيحي للنوع أكثر تحديدًا باستخدام اتحاد من الأنواع الحرفية:

```
const v: { x: 1 | 2 | 3 } = {  
  x: 1, // x is now a union of literal types: 1 | 2 | 3  
};  
v.x = 3; // Valid  
v.x = 100; // Invalid
```

تضييق النوع

تضييق الأنواع هو العملية التي يُضَيَّق فيها نوع عام إلى نوع أكثر تحديدًا في الشيفرة وتحدد أن TypeScript ويحدث ذلك عندما تحلل TypeScript. شروطًا أو عمليات معينة يمكنها تحسين معلومات النوع.

يمكن أن يحدث تضيق الأنواع بطرق مختلفة، منها:

الشروط

TypeScript يمكن لـ `switch` أو `if` باستخدام العبارات الشرطية، مثل تضيق النوع استنادًا إلى نتيجة الشرط. على سبيل المثال:

```
let x: number | undefined = 10;

if (x !== undefined) {
  x += 100; // The type is number, which had been narrowed by
}
```

طرح استثناء أو الإرجاع

TypeScript يمكن استخدام طرح خطأ أو الإرجاع المبكر من فرع لمساعدة على تضيق النوع. على سبيل المثال:

```
let x: number | undefined = 10;

if (x === undefined) {
  throw 'error';
}
x += 100;
```

ما يلي TypeScript تشمل الطرق الأخرى لتضيق الأنواع في:

- يُستخدم للتحقق مما إذا كان كائن ما مثيلاً لصنف `instanceof`: المعامل محدد.
- يُستخدم للتحقق مما إذا كانت خاصية موجودة في كائن `in`: المعامل.
- يُستخدم للتحقق من نوع قيمة في وقت التشغيل `typeof`: المعامل.
- تُستخدم للتحقق مما إذا كانت `Array.isArray()` الدوال المضمنة مثل القيمة مصفوفة.

الاتحاد المميّز

تُضاف فيه “علامة” TypeScript يُعد استخدام “الاتحاد المميّز” نمطًا في صريحة إلى الكائنات للتمييز بين الأنواع المختلفة داخل اتحاد. ويُشار إلى هذا النمط أيضًا باسم “الاتحاد الموسوم”. في المثال التالي، تمثل الخاصية “type” هذه “العلامة”:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
  switch (input.type) {
    case 'type_a':
      return input.value + 100; // type is A
    case 'type_b':
      return input.value + 'extra'; // type is B
  }
};
```

حراس الأنواع المعرّفة من قبل المستخدم

من تحديد نوع، يمكن كتابة دالة TypeScript في الحالات التي لا تتمكن فيها مساعدة تُعرف باسم “حارس نوع معرّف من قبل المستخدم”. في المثال التالي، سنستخدم مُستد نوع لتضييق النوع بعد تطبيق عملية تصفية معينة:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x !== null); // The type is (string |

const isValid = (item: string | null): item is string => item !=

const r2 = data.filter(isValid); // The type is fine now string,
```

Switch-true بالتضييق باستخدام

ما يتيح لك switch-true، switch-true التضييق باستخدام TypeScript 5.3 تضيف باستخدام شروط switch (true) الفوضوية بـ if/else استبدال سلاسل منطقية. ويحسن ذلك قابلية القراءة مع استمرار تضييق الأنواع. وهو يشبه مطابقة الأنماط، لكنه أبسط.

```
function classify(x: unknown) {
  switch (true) {
    case typeof x === 'string':
      return `${x.toUpperCase()}`;
    case typeof x === 'number':
      return x > 0 ? 'positive' : 'negative';
    case Array.isArray(x):
      return `[${x.length} items]`;
    default:
      return 'something else';
  }
}
```

الأنواع الأولية

سبعة أنواع أولية. يشير نوع البيانات الأولي إلى نوع ليس TypeScript يدعم تكون جميع الأنواع الأولية، TypeScript كائنًا ولا ترتبط به أي أساليب. في غير قابلة للتغيير، ما يعني أنه لا يمكن تغيير قيمها بعد إسنادها.

string

البيانات النصية، وتكون القيمة دائمًا محاطة string يخزن النوع البدائي بعلامتي اقتباس مزدوجتين أو مفردتين.

```
const x: string = 'x';
const y: string = 'y';
```

يمكن أن تمتد السلاسل النصية عبر عدة أسطر إذا أُحيطت بعلامة الفاصلة (,) العليا المائلة:

```
let sentence: string = `xxx,
  yyy`;
```

boolean

وإما true قيمة ثنائية، إما TypeScript في boolean يخزن نوع البيانات false.

```
const isReady: boolean = true;
```

number

بقائمة فاصلة عائمة ذات 64 بت. TypeScript في number يُمثّل نوع البيانات TypeScript تمثيل الأعداد الصحيحة والكسور. يدعم number ويمكن لنوع: أيضًا النظام السداسي عشري والثنائي والثماني، على سبيل المثال

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal starts with 0x
const binary: number = 0b1010; // Binary starts with 0b
const octal: number = 0o633; // Octal starts with 0o
```

bigint

قيمًا صحيحة يمكن أن تكون أكبر من الحد الأقصى للعدد bigint يمثّل وهو $53^2 - 1$ ، number الصحيح الآمن الذي يدعمه.

إلى نهاية n أو بإضافة BigInt() باستدعاء الدالة المضمّنة bigint يمكن إنشاء: أي قيمة عددية صحيحة حرفية

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

ملاحظات:

- ولا يمكن استخدامها مع الكائن number، مع bigint لا يمكن مزج قيم
- ؛ بل يجب تحويلها قسراً إلى النوع نفسه Math المضمن
- أو إصداراً أحدث ES2020 إلا إذا كان إعداد الهدف bigint لا تتوفر قيم

Symbol

الرموز هي معرفّات فريدة يمكن استخدامها مفاتيح للخصائص في الكائنات لمنع تعارض الأسماء.

```
type Obj = {
  [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null وundefined

كلاهما انعدام القيمة أو غياب أي قيمة undefined و null يمثل النوعان

أن القيمة لم تُسند أو تُهيأ، أو يشير إلى غياب غير undefined يعني النوع مقصود للقيمة.

أننا نعلم أن الحقل لا يحتوي على قيمة، ومن ثم تكون القيمة null يعني النوع غير متاحة، وهو يشير إلى غياب مقصود للقيمة.

Array

هي نوع بيانات يمكنه تخزين عدة قيم من النوع نفسه أو من array المصفوفة:
أنواع مختلفة. ويمكن تعريفها باستخدام الصياغة الآتية:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

المصفوفات المخصصة للقراءة فقط باستخدام الصياغة TypeScript يدعم الآتية:

```
const x: readonly string[] = ['a', 'b']; // Readonly modifier
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalid
```

المخصصة للقراءة فقط Tuple وأنواع Tuple TypeScript يدعم:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

حرفيًا «أي» قيمة، وهو النوع الافتراضي عندما يتعذر any يمثل نوع البيانات استدلال النوع أو عندما لا يكون محددًا TypeScript على.

التحقق من الأنواع، ولذلك لا TypeScript يتخطى مصرّف any عند استخدام لإسكات any وبوجه عام، لا تستخدم any. توجد سلامة للأنواع عند استخدام المصرّف عند حدوث خطأ؛ بل ركّز بدلاً من ذلك على إصلاح الخطأ، لأن يجعل من الممكن خرق العقود وفقدان فوائد الإكمال التلقائي any استخدام في TypeScript.

إلى JavaScript مفيدًا أثناء الانتقال التدريجي من any قد يكون النوع TypeScript، إذ يمكنه إسكات المصرّف.

الذي TypeScript noImplicitAny للمشروعات الجديدة، استخدم إعداد any من إصدار أخطاء في المواضيع التي يُستخدم فيها TypeScript يمكن

يُستدل عليه.

مصدرًا للأخطاء التي يمكنها إخفاء مشكلات حقيقية any عادةً ما يكون النوع في أنواعك. تجنّب استخدامه قدر الإمكان.

التعليقات التوضيحية للأنواع

var و let يمكن اختيارياً إضافة نوع إلى المتغيرات المصرّح بها باستخدام const:

```
const x: number = 1;
```

استدلال الأنواع، ولا سيما الأنواع البسيطة، لذلك لا تكون TypeScript جيد هذه التصريحات ضرورية في معظم الحالات.

يمكن إضافة تعليقات توضيحية للأنواع إلى معاملات الدوال:

```
function sum(a: number, b: number) {  
    return a + b;  
}
```

(فيما يأتي مثال يستخدم دالة مجهولة (تُسمّى أيضاً دالة لامبدا

```
const sum = (a: number, b: number) => a + b;
```

يمكن الاستغناء عن هذه التعليقات التوضيحية عند وجود قيمة افتراضية للمعامل:

```
const sum = (a = 10, b: number) => a + b;
```

يمكن إضافة تعليقات توضيحية لنوع الإرجاع إلى الدوال:

```
const sum = (a = 10, b: number): number => a + b;
```

يفيد هذا خصوصاً مع الدوال الأكثر تعقيداً، إذ يمكن أن تساعدك كتابة نوع الإرجاع قبل التنفيذ على التفكير ملياً في الدالة.

بوجه عام، فُكِّر في إضافة تعليقات توضيحية إلى مواقع الأنواع، لا إلى المتغيرات المحلية داخل المتن، وأضف دائمًا أنواعًا إلى الكائنات الحرفية.

الخصائص الاختيارية

يمكن للكائن تحديد خصائص اختيارية بإضافة علامة استفهام ? إلى نهاية اسم الخاصية:

```
type X = {  
  a: number;  
  b?: number; // Optional  
};
```

يمكن تحديد قيمة افتراضية عندما تكون الخاصية اختيارية:

```
type X = {  
  a: number;  
  b?: number;  
};  
const x = ({ a, b = 100 }: X) => a + b;
```

الخصائص المخصصة للقراءة فقط

الذي يضمن عدم readonly، يمكن منع الكتابة إلى خاصية باستخدام المعدّل إمكانية إعادة كتابة الخاصية، لكنه لا يوفر أي ضمان بعدم قابلية التغيير الكاملة:

```
interface Y {  
  readonly a: number;  
}
```

```
type X = {  
  readonly a: number;  
};
```

```
type J = Readonly<{  
  a: number;  
}>
```

```
>;
```

```
type K = {  
  readonly [index: number]: string;  
};
```

تواقيع الفهرسة

كتواقيع فهرسة symbol و number و string يمكننا استخدام TypeScript، في

```
type K = {  
  [name: string | number]: string;  
};  
const k: K = { x: 'x', 1: 'b' };  
console.log(k['x']);  
console.log(k[1]);  
console.log(k['1']); // Same result as k[1]
```

إلى number يحوّل تلقائيًا فهرسًا من النوع JavaScript يرجى ملاحظة أن القيمة نفسها k["1"] أو k[1] ولذلك يُرجع string، فهرس من النوع

توسعة الأنواع

(بنسخ الأعضاء من نوع آخر) interface يمكن توسيع

```
interface X {  
  a: string;  
}  
interface Y extends X {  
  b: string;  
}
```

يمكن أيضًا التوسيع من أنواع متعددة

```
interface A {  
  a: string;  
}
```

```
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

فقط مع الواجهات والفئات، أما الأنواع extends تعمل الكلمة المفتاحية فتستخدم التقاطع:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

يمكن توسيع نوع باستخدام واجهة، ولكن ليس العكس:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

الأنواع الحرفية

النوع الحرفي هو مجموعة أحادية العنصر داخل نوع جماعي؛ فهو يعرف قيمة JavaScript دقيقة جدًا تكون قيمة بدائية في

هي الأعداد والسلاسل النصية والقيم TypeScript الأنواع الحرفية في المنطقية.

مثال على القيم الحرفية:

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

تُستخدم الأنواع الحرفية النصية والعديدية والمنطقية في الاتحادات وحراس الأنواع والأسماء المستعارة للأنواع. في المثال الآتي، يمكنك رؤية اسم من القيم المحددة فقط، ولا تكون أي سلسلة 0 مستعار لنوع اتحاد. يتكون نصية أخرى صالحة:

```
type O = 'a' | 'b' | 'c';
```

استدلال الأنواع الحرفية

تسمح باستدلال نوع متغير أو TypeScript استدلال الأنواع الحرفية ميزة في معامل استنادًا إلى قيمته.

نوعًا حرفيًا لأن القيمة x يعدّ TypeScript في المثال الآتي، يمكننا أن نرى أن string بوصفه من النوع y لا يمكن تغييرها في أي وقت لاحق، بينما يُستنتج لأنه يمكن تعديله في أي وقت لاحق.

```
const x = 'x'; // Literal type of 'x', because this value cannot
let y = 'y'; // Type string, as we can change this value
```

لا قيمة حرفية) string استنتج بوصفه x في المثال الآتي، يمكننا أن نرى أن يعتبر أن القيمة يمكن تغييرها في أي وقت لاحق TypeScript لأن a من

```
type X = 'a' | 'b';
```

```
let o = {
  x: 'a', // This is a wider string
};
```

```
const fn = (x: X) => `${x}-foo`;
```

```
console.log(fn(o.x)); // Argument of type 'string' is not assignable
```

نوع أضيق X لأن fn إلى o.x كما ترى، تطرح الشيفرة خطأ عند تمرير

x: أو النوع const يمكننا حل هذه المشكلة باستخدام توكيد نوع مع

```
let o = {  
  x: 'a' as const,  
};
```

أو:

```
let o = {  
  x: 'a' as X,  
};
```

strictNullChecks

يفرض تحققًا صارمًا من TypeScript هو خيار لمصرّف strictNullChecks إلى undefined أو null القيم الخالية. عند تمكين هذا الخيار، لا يمكن إسناد المتغيرات والمعاملات إلا إذا صُرح بوضوح بأنها من ذلك النوع باستخدام نوع وإذا لم يُصرّح بوضوح بأن متغيرًا أو معاملًا يقبل null | undefined. الاتحاد خطأ لمنع أخطاء وقت التشغيل المحتملة TypeScript قيمة خالية، فسيولّد

التعدادات

مجموعة من القيم الثابتة المسماة enum يمثل TypeScript، في

```
enum Color {  
  Red = '#ff0000',  
  Green = '#00ff00',  
  Blue = '#0000ff',  
}
```

يمكن تعريف التعدادات بطرائق مختلفة:

التعدادات العددية

التعداد العددي هو تعداد تُسَدّ فيه قيمة عددية إلى كل TypeScript، في ثابت، بدءًا من 0 افتراضيًا.

```
enum Size {  
    Small, // value starts from 0  
    Medium,  
    Large,  
}
```

:يمكن تحديد قيم مخصّصة بإسنادها صراحةً

```
enum Size {  
    Small = 10,  
    Medium,  
    Large,  
}  
console.log(Size.Medium); // 11
```

التعدادات النصية

التعداد النصي هو تعداد تُسَدّ فيه قيمة نصية إلى كل ثابت TypeScript، في

```
enum Language {  
    English = 'EN',  
    Spanish = 'ES',  
}
```

باستخدام تعدادات غير متجانسة يمكن أن TypeScript ملاحظة: يسمح تتعايش فيها الأعضاء النصية والعددية.

التعدادات الثابتة

هو نوع خاص من التعدادات تكون فيه جميع TypeScript التعداد الثابت في القيم معروفة في وقت التصريف وتُضمّن مباشرةً في كل موضع يُستخدم فيه

التعداد، ما ينتج شيفرة أكثر كفاءة.

```
const enum Language {  
    English = 'EN',  
    Spanish = 'ES',  
}  
console.log(Language.English);
```

سيُصرَّف إلى:

```
console.log('EN' /* Language.English */);
```

ملاحظات:

تحتوي التعدادات الثابتة على قيم مضمنة في الشيفرة، ما يؤدي إلى محو التعداد، ويمكن أن يكون ذلك أكثر كفاءة في المكتبات المستقلة، لكنه غير مرغوب فيه بوجه عام. كذلك لا يمكن أن تحتوي التعدادات الثابتة على أعضاء محسوبة.

الربط العكسي

إلى القدرة على استرداد TypeScript يشير الربط العكسي في تعدادات اسم عضو التعداد من قيمته. توجد لأعضاء التعداد افتراضياً روابط أمامية من الاسم إلى القيمة، ولكن يمكن إنشاء روابط عكسية بتحديد قيم كل عضو صراحةً. تفيد الروابط العكسية عندما تحتاج إلى البحث عن عضو تعداد باستخدام قيمته، أو عندما تحتاج إلى التكرار على جميع أعضاء التعداد. لاحظ أن أعضاء التعداد العددية وحدها هي التي تولد روابط عكسية، بينما لا يُولَّد أي رابط عكسي لأعضاء التعداد النصية.

التعداد الآتي:

```
enum Grade {  
    A = 90,  
    B = 80,  
    C = 70,
```

```
    F = 'fail',  
  }
```

يُصَرَّف إلى:

```
'use strict';  
var Grade;  
(function (Grade) {  
    Grade[(Grade['A'] = 90)] = 'A';  
    Grade[(Grade['B'] = 80)] = 'B';  
    Grade[(Grade['C'] = 70)] = 'C';  
    Grade['F'] = 'fail';  
})(Grade || (Grade = {}));
```

لذلك، يعمل ربط القيم بالمفاتيح مع أعضاء التعداد العددية، ولكن ليس مع أعضاء التعداد النصية:

```
enum Grade {  
    A = 90,  
    B = 80,  
    C = 70,  
    F = 'fail',  
}  
const myGrade = Grade.A;  
console.log(Grade[myGrade]); // A  
console.log(Grade[90]); // A  
  
const failGrade = Grade.F;  
console.log(failGrade); // fail  
console.log(Grade[failGrade]); // Element implicitly has an 'any'
```

التعدادات المحيطة

هو نوع من التعدادات يُعرَّف في ملف TypeScript التعداد المحيط في من دون تنفيذ مرتبط به. ويتيح لك تعريف مجموعة من (*.d.ts) تصريح

الثوابت المسماة التي يمكن استخدامها بطريقة آمنة من ناحية الأنواع عبر ملفات مختلفة من دون الحاجة إلى استيراد تفاصيل التنفيذ في كل ملف.

الأعضاء المحسوبة والثابتة

العضو المحسوب هو عضو في تعداد تُحسب قيمته في TypeScript، في وقت التشغيل، بينما العضو الثابت هو عضو تُحدّد قيمته في وقت التصريف ولا يمكن تغييرها في أثناء وقت التشغيل. يُسمح بالأعضاء المحسوبة في التعدادات العادية، بينما يُسمح بالأعضاء الثابتة في كل من التعدادات العادية والثابتة.

```
// Constant members
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 generation at compilation time

// Computed members
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // random number generated at run time
```

يُعبّر عن التعدادات باتحادات تتألف من أنواع أعضائها. ويمكن تحديد قيم كل عضو من خلال تعبيرات ثابتة أو غير ثابتة، مع إسناد أنواع حرفية إلى الأعضاء E.A وأنواعه الفرعية E التي تمتلك قيمًا ثابتة. للتوضيح، تأمل تصريح النوع E.A | E.B | E.C الاتحاد E في هذه الحالة، يمثّل E.C وE.B وE.A.

```
const identity = (value: number) => value;
```

```
enum E {
    A = 2 * 5, // Numeric literal
```

```

    B = 'bar', // String literal
    C = identity(42), // Opaque computed
  }

console.log(E.C); //42

```

التضييق

هو عملية تنقيح نوع متغير داخل كتلة شرطية. TypeScript تضييق الأنواع في ويفيد ذلك عند التعامل مع أنواع الاتحاد، حيث يمكن أن يكون للمتغير أكثر من نوع واحد.

على عدة طرائق لتضييق النوع TypeScript يتعرّف:

typeof حراس الأنواع باستخدام

يتحقق من نوع TypeScript هو حارس نوع محدد في typeof حارس النوع المضمّن الخاص به JavaScript متغير استنادًا إلى نوع.

```

const fn = (x: number | string) => {
  if (typeof x === 'number') {
    return x + 1; // x is number
  }
  return -1;
};

```

التضييق بحسب الصدقية

من خلال التحقق مما إذا TypeScript يعمل التضييق بحسب الصدقية في كان المتغير صادقًا أم زائفًا لتضييق نوعه وفقًا لذلك.

```

const toUpperCase = (name: string | null) => {
  if (name) {
    return name.toUpperCase();
  } else {
    return null;
  }
};

```

```
    }  
};
```

التضييق بحسب المساواة

من خلال التحقق مما إذا TypeScript يعمل التضييق بحسب المساواة في كان المتغير يساوي قيمة محددة أم لا، لتضييق نوعه وفقًا لذلك.

ومعاملات المساواة مثل === و !== و switch يُستخدم بالتزامن مع عبارات و != لتضييق الأنواع.

```
const checkStatus = (status: 'success' | 'error') => {  
  switch (status) {  
    case 'success':  
      return true;  
    case 'error':  
      return null;  
  }  
};
```

In التضييق باستخدام المعامل

هو طريقة لتضييق نوع متغير TypeScript في in التضييق باستخدام المعامل استنادًا إلى وجود خاصية ضمن نوع المتغير.

```
type Dog = {  
  name: string;  
  breed: string;  
};
```

```
type Cat = {  
  name: string;  
  likesCream: boolean;  
};
```

```
const getAnimalType = (pet: Dog | Cat) => {  
  if ('breed' in pet) {
```

```

        return 'dog';
    } else {
        return 'cat';
    }
};

```

instanceof التضييق باستخدام

هو طريقة لتضييق TypeScript في instanceof التضييق باستخدام المعامل نوع متغير استنادًا إلى دالة المُنشئ الخاصة به، عن طريق التحقق مما إذا كان الكائن مثيلًا لفئة أو واجهة معينة.

```

class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50

```

الإسنادات

باستخدام الإسنادات هو طريقة لتضييق نوع TypeScript تضييق الأنواع في متغير استنادًا إلى القيمة المسندة إليه. عندما تُسند قيمة إلى متغير، يستنتج

نوعه استنادًا إلى القيمة المسندة، ويضيق نوع المتغير ليتطابق TypeScript مع النوع المستنتج.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

تحليل تدفق التحكم

هو طريقة لتحليل تدفق الشيفرة تحليلًا TypeScript تحليل تدفق التحكم في ساكنًا لاستدلال أنواع المتغيرات، ما يسمح للمصرّف بتضييق أنواع تلك المتغيرات حسب الحاجة استنادًا إلى نتائج التحليل.

كان تحليل تدفق الشيفرة يُطبَّق فقط على الشيفرة، TypeScript 4.4 قبل يمكن تطبيقه أيضًا على TypeScript 4.4 ولكن بدءًا من if، داخل عبارة التعابير الشرطية والوصول إلى الخصائص المميّزة المشار إليها بصورة غير const. مباشرة من خلال متغيرات.

على سبيل المثال:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
    const isFoo = obj.kind === 'foo';
}
```

```

const isFoo = obj.kind === 'foo' ,
if (isFoo) {
    obj.foo;
} else {
    obj.bar;
}
};

```

بعض الأمثلة التي لا يحدث فيها التضييق:

```

const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // Error, no narrowing because isString it is
    }
};

```

```

const f6 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Error, no narrowing because obj is assigned
    }
};

```

ملاحظات: يُحلَّل ما يصل إلى خمسة مستويات من الإحالة غير المباشرة في التعبيرات الشرطية.

مסندات الأنواع

هي دوال تُرجع قيمة منطقية وتُستخدم TypeScript مسندات الأنواع في لتضييق نوع متغير إلى نوع أكثر تحديدًا.

```

const isString = (value: unknown): value is string => typeof va

```

```
const foo = (bar: unknown) => {
  if (isString(bar)) {
    console.log(bar.toUpperCase());
  } else {
    console.log('not a string');
  }
};
```

في دوال مثل (T is x) مسندات الأنواع تلقائيًا (مثل TypeScript 5.5) ينتج ما يوفر أنواعًا أدق، undefined، ولذلك يعرف متى تُزال قيم مثل filter. x !== undefined وأخطاء أقل. ويعمل هذا مع عمليات التحقق الواضحة (مثل x!! ولكن ليس مع العمليات الملتبسة مثل undefined)

```
const nums = [1, null, 2].filter(x => x !== null);
```

الاتحادات المميّزة

هي نوع من أنواع الاتحاد يستخدم خاصية TypeScript الاتحادات المميّزة في مشتركة، تُعرف باسم المميّز، لتضييق مجموعة الأنواع الممكنة للاتحاد.

```
type Square = {
  kind: 'square'; // Discriminant
  size: number;
};
```

```
type Circle = {
  kind: 'circle'; // Discriminant
  radius: number;
};
```

```
type Shape = Square | Circle;
```

```
const area = (shape: Shape) => {
  switch (shape.kind) {
    case 'square':
      return Math.pow(shape.size, 2);
  }
};
```

```

        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

```

```

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

```

```

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172

```

النوع never

عندما يُضَيَّق متغير إلى نوع لا يمكن أن يحتوي على أي قيم، يستنتج مصرّف ويرجع ذلك إلى أن `never`. أن المتغير لا بد أن يكون من النوع TypeScript. يمثل قيمة لا يمكن إنتاجها مطلقًا `never` النوع.

```

const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // val has type never here because it can never be anything
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};

```

التحقق من الشمول

تضمن معالجة جميع الحالات TypeScript التحقق من الشمول ميزة في `if` أو عبارة `switch` الممكنة لاتحاد مميّز في عبارة.

```

type Direction = 'up' | 'down';

```



```
const move = (direction: Direction) => {
  switch (direction) {
    case 'up':
      console.log('Moving up');
      break;
    case 'down':
      console.log('Moving down');
      break;
    default:
      const exhaustiveCheck: never = direction;
      console.log(exhaustiveCheck); // This line will never
  }
};
```

لضمان أن الحالة الافتراضية مستوفية لجميع `never` يُستخدم النوع سيُصدر خطأ إذا أُضيفت قيمة جديدة إلى النوع TypeScript الاحتمالات، وأن `switch` من دون معالجتها في عبارة `Direction`.

أنواع الكائنات

تصف أنواع الكائنات شكل الكائن. فهي تحدد أسماء TypeScript في خصائص الكائن وأنواعها، وما إذا كانت تلك الخصائص مطلوبة أم اختيارية.

يمكنك تعريف أنواع الكائنات بطريقتين أساسيتين في TypeScript:

تعرف الواجهة شكل الكائن بتحديد أسماء خصائصه وأنواعها وما إذا كانت اختيارية.

```
interface User {
  name: string;
  age: number;
  email?: string;
}
```

يعرّف الاسم المستعار للنوع، على غرار الواجهة، شكل الكائن. لكنه يستطيع أيضًا إنشاء نوع مخصّص جديد يستند إلى نوع موجود أو إلى مجموعة من

الأنواع الموجودة. ويشمل ذلك تعريف أنواع الاتحاد وأنواع التقاطع وأنواع معقدة أخرى.

```
type Point = {  
  x: number;  
  y: number;  
};
```

يمكن أيضًا تعريف نوع بصورة مجهولة:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;  
console.log(sum({ a: 5, b: 1 }));
```

(مجهول الاسم) Tuple نوع

هو نوع يمثل مصفوفة ذات عدد ثابت من العناصر وأنواعها Tuple نوع عددًا محددًا من العناصر وأنواعًا محددة لكل منها Tuple المقابلة. يفرض نوع عندما تريد تمثيل مجموعة من القيم ذات Tuple بترتيب ثابت. تفيد أنواع أنواع محددة، بحيث يكون لموضع كل عنصر في المصفوفة معنى محدد.

```
type Point = [number, number];
```

(المسمى (الموسوم) Tuple نوع

تسميات أو أسماء اختيارية لكل عنصر. هذه Tuple يمكن أن تتضمن أنواع التسميات مخصصة لتحسين قابلية القراءة والمساعدة التي تقدمها الأدوات، ولا تؤثر في العمليات التي يمكنك إجراؤها عليها.

```
type T = string;  
type Tuple1 = [T, T];  
type Tuple2 = [a: T, b: T];  
type Tuple3 = [a: T, T]; // Named Tuple plus Anonymous Tuple
```

ثابت الطول Tuple

يفرض عددًا ثابتًا من العناصر Tuple ثابت الطول هو نوع محدد من Tuple بعد تعريفه Tuple ذات أنواع محددة، ولا يسمح بأي تعديلات على طول

ثابتة الطول عندما تحتاج إلى تمثيل مجموعة من القيم بعدد Tuple تفيد أنواع Tuple محدد من العناصر وأنواع محددة، وتريد ضمان عدم تغيير طول وأنواعه من دون قصد.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

نوع الاتحاد

نوع الاتحاد هو نوع يمثل قيمة يمكن أن تكون أحد عدة أنواع. ويُعبّر عن أنواع الاتحاد باستخدام الرمز | بين كل نوع ممكن.

```
let x: string | number;
x = 'hello'; // Valid
x = 123; // Valid
```

أنواع التقاطع

نوع التقاطع هو نوع يمثل قيمة تمتلك جميع خصائص نوعين أو أكثر. ويُعبّر عن أنواع التقاطع باستخدام الرمز & بين كل نوع

```
type X = {
  a: string;
};
```

```
type Y = {
  b: string;
};
```

```
type J = X & Y; // Intersection
```

```
const j: J = {
```

```
a: 'a',  
b: 'b',  
};
```

فهرسة الأنواع

تشير فهرسة الأنواع إلى القدرة على تعريف أنواع يمكن فهرستها بمفتاح غير معروف مسبقًا، باستخدام توقيع فهرسة لتحديد نوع الخصائص غير المصرح بها صراحةً.

```
type Dictionary<T> = {  
  [key: string]: T;  
};  
const myDict: Dictionary<string> = { a: 'a', b: 'b' };  
console.log(myDict['a']); // Returns a
```

النوع من القيمة

إلى الاستدلال التلقائي لنوع من TypeScript يشير «النوع من القيمة» في قيمة أو تعبير من خلال استدلال النوع.

```
const x = 'x'; // TypeScript infers 'x' as a string literal with
```

النوع من القيمة المرجعة للدالة

يشير «النوع من القيمة المرجعة للدالة» إلى القدرة على استدلال نوع تحديد نوع TypeScript الإرجاع لدالة تلقائيًا استنادًا إلى تنفيذها. ويتيح ذلك لـ القيمة التي تُرجعها الدالة من دون تعليقات توضيحية صريحة للنوع.

```
const add = (x: number, y: number) => x + y; // TypeScript can
```

النوع من الوحدة

يشير «النوع من الوحدة» إلى القدرة على استخدام القيم التي تصدرها الوحدة لاستدلال أنواعها تلقائيًا. عندما تصدر وحدة قيمة من نوع محدد، يمكن استخدام تلك المعلومات لاستدلال نوع تلك القيمة تلقائيًا عند TypeScript لاستيرادها إلى وحدة أخرى.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r is number
```

الأنواع المعينة

إنشاء أنواع جديدة استنادًا إلى نوع TypeScript تتيح لك الأنواع المعيّنة في موجود، وذلك بتحويل كل خاصية باستخدام دالة تعيين. ومن خلال تعيين الأنواع الموجودة، يمكنك إنشاء أنواع جديدة تمثّل المعلومات نفسها بتنسيق مختلف. ولإنشاء نوع معيّن، تصل إلى خصائص نوع موجود باستخدام المعامل ثم تعدّلها لإنتاج نوع جديد. في المثال الآتي `keyof`

```
type MappedType<T> = {
  [P in keyof T]: T[P][];
};

type MyType = {
  foo: string;
  bar: number;
};

type MyNewType = MappedType<MyType>;
const x: MyNewType = {
  foo: ['hello', 'world'],
  bar: [1, 2, 3],
};
```

وإنشاء نوع جديد تكون فيه كل T ، لتعيين خصائص `MyMappedType` نعرّف `MyNewType` خاصة مصفوفةً من نوعها الأصلي. وباستخدام ذلك، ننشئ

ولكن مع جعل كل خاصية `MyType` لتمثيل المعلومات نفسها الموجودة في مصفوفة.

معدّلات الأنواع المعيّنة

تحويل الخصائص داخل نوع TypeScript تتيح معدّلات الأنواع المعيّنة في موجود:

- `readonly` أو `+readonly`: مخصّصة للقراءة فقط.
- `-readonly`: يجعل الخاصية في النوع المعيّن قابلة للتغيير.
- `?`: يجعل الخاصية في النوع المعيّن اختيارية.

أمثلة:

```
type Readonly<T> = { readonly [P in keyof T]: T[P] }; // All properties are read-only
type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // All properties are mutable
type MyPartial<T> = { [P in keyof T]?: T[P] }; // All properties are optional
```

الأنواع الشرطية

الأنواع الشرطية هي طريقة لإنشاء نوع يعتمد على شرط، حيث يُحدّد النوع المراد إنشاؤه استنادًا إلى نتيجة الشرط. وتُعرّف باستخدام الكلمة المفتاحية `extends` والمعامل الثلاثي للاختيار المشروط بين نوعين.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type false
```

الأنواع الشرطية التوزيعية

الأنواع الشرطية التوزيعية هي ميزة تسمح بتوزيع نوع على اتحاد من الأنواع من خلال تطبيق تحويل على كل عضو في الاتحاد على حدة. يمكن أن يكون هذا مفيدًا خصوصًا عند التعامل مع الأنواع المعيّنة أو الأنواع عالية الرتبة.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number /
```

في الأنواع الشرطية infer استدلال النوع باستخدام

في الأنواع الشرطية لاستدلال (استخراج) infer تُستخدم الكلمة المفتاحية نوع معامل عام من نوع يعتمد عليه. ويتيح لك ذلك كتابة تعريفات أنواع أكثر مرونة وقابلية لإعادة الاستخدام.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

الأنواع الشرطية المعرّفة مسبقًا

الأنواع الشرطية المعرّفة مسبقًا هي أنواع شرطية مضمّنة في TypeScript، توفرها اللغة. وقد صُممت لتنفيذ تحويلات شائعة للأنواع استنادًا إلى خصائص نوع معيّن.

Exclude<UnionType, ExcludedType>: جميع Type يزيل هذا النوع من ExcludedType الأنواع القابلة للإسناد إلى

Extract<Type, Union>: جميع الأنواع القابلة Union يستخرج هذا النوع من Type للإسناد إلى

`Nullable<Type>`: `null` و `undefined` من `Type`.

`ReturnType<Type>`: `Type` يستخرج هذا النوع نوع الإرجاع لنوع دالة.

`Parameters<Type>`: `Type` يستخرج هذا النوع أنواع معاملات نوع دالة.

`Required<Type>`: `Type` يجعل هذا النوع جميع الخصائص في.

`Partial<Type>`: `Type` اختيارية يجعل هذا النوع جميع الخصائص في.

`Readonly<Type>`: `Type` مخصصة يجعل هذا النوع جميع الخصائص في للقراءة فقط.

أنواع اتحادات القوالب

يمكن استخدام أنواع اتحادات القوالب لدمج النصوص ومعالجتها داخل نظام الأنواع، على سبيل المثال:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active"
```

النوع Any

هو نوع خاص (نوع فائق عام) يمكن استخدامه لتمثيل أي نوع من `any` النوع القيم (الأنواع الأولية، والكائنات، والمصفوفات، والدوال، والأخطاء، والرموز). وغالبًا ما يُستخدم في الحالات التي لا يكون فيها نوع القيمة معروفًا في وقت التصريف، أو عند التعامل مع قيم من واجهات برمجة تطبيقات أو مكتبات `TypeScript` خارجية لا تحتوي على تعريفات أنواع.

بأن القيم يجب أن تُمثل `TypeScript` فإنك تُعلم مصرّف `any` باستخدام النوع من دون أي قيود. لتحقيق أقصى قدر من أمان الأنواع في شيفرتك، ضع ما يلي في الحسبان:

- في الحالات المحددة التي يكون فيها النوع مجهولاً any احصر استخدام بالفعل.
- من دالة، لأن ذلك يُضعف أمان الأنواع في الشيفرة التي any لا تُعد أنواع تستخدمها.
- إذا كنت بحاجة إلى إسكات المصّرّف @ts-ignore استخدم any بدلاً من

```
let value: any;
value = true; // Valid
value = 7; // Valid
```

Unknown النوع

قيمةً من نوع مجهول. وعلى خلاف unknown يمثل النوع في TypeScript، فحصاً للنوع أو unknown الذي يسمح بأي نوع من القيم، يتطلب any النوع توكيداً له قبل أن يمكن استخدامه بطريقة محددة، ولذلك لا يُسمح بإجراء أي من دون توكيد نوع أكثر تحديداً أو unknown عمليات على قيمة من النوع. تضيق النوع إليه أولاً

نفسه، وهو بديل آمن unknown وإلى any إلا إلى unknown لا يمكن إسناد النوع any من ناحية الأنواع للنوع.

```
let value: unknown;

let value1: unknown = value; // Valid
let value2: any = value; // Valid
let value3: boolean = value; // Invalid
let value4: number = value; // Invalid

const add = (a: unknown, b: unknown): number | undefined =>
  typeof a === 'number' && typeof b === 'number' ? a + b : undefined
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Void النوع

للإشارة إلى أن الدالة لا تُعيد قيمة void يُستخدم النوع.

```
const sayHello = (): void => {  
    console.log('Hello!');  
};
```

Never نوع

القيم التي لا تحدث مطلقًا. ويُستخدم للدلالة على الدوال أو never يمثل النوع التعبيرات التي لا تعود مطلقًا أو ترمي خطأ.

على سبيل المثال، حلقة لا نهائية:

```
const infiniteLoop = (): never => {  
    while (true) {  
        // do something  
    }  
};
```

رمي خطأ:

```
const throwError = (message: string): never => {  
    throw new Error(message);  
};
```

مفيد في ضمان أمان الأنواع واكتشاف الأخطاء المحتملة في never النوع على تحليل أنواع أكثر دقة واستدلالها عند TypeScript شيفرتك. فهو يساعد استخدامه مع أنواع أخرى وتعليمات تدفق التحكم، على سبيل المثال:

```
type Direction = 'up' | 'down';  
const move = (direction: Direction): void => {  
    switch (direction) {  
        case 'up':  
            // move up  
            break;  
        case 'down':  
            // move down  
            break;  
    }  
};
```

```

        break;
    default:
        const exhaustiveCheck: never = direction;
        throw new Error(`Unhandled direction: ${exhaustiveCheck}`);
    }
};

```

الواجهة والنوع

الصياغة الشائعة

تحدد الواجهات بنية الكائنات، مع تعيين أسماء الخصائص أو TypeScript، في الأساليب التي يجب أن يحتوي عليها الكائن وأنواعها. والصياغة الشائعة هي كما يلي TypeScript لتعريف واجهة في:

```

interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}

```

وبالمثل لتعريف نوع:

```

type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};

```

يحدد اسم الواجهة: `interface InterfaceName` أو `type TypeName`.
يحدد خصائص الواجهة مع الأنواع المقابلة لها. يمكن `property1: Type1`.
تعريف خصائص متعددة، تفصل بين كل منها فاصلة منقوطة `method1(arg1: ArgType1, arg2: ArgType2): ReturnType`. تُعرّف `;` يحدد أساليب الواجهة.

الأساليب بأسمائها، تليها قائمة معاملات بين قوسين ثم نوع الإرجاع. يمكن تعريف أساليب متعددة، تفصل بين كل منها فاصلة منقوطة.

مثال على واجهة:

```
interface Person {  
    name: string;  
    age: number;  
    greet(): void;  
}
```

مثال على نوع:

```
type TypeName = {  
    property1: string;  
    method1(arg1: string, arg2: string): string;  
};
```

تُستخدم الأنواع لتحديد شكل البيانات وفرض التحقق من TypeScript، في تبعًا لحالة TypeScript، الأنواع. توجد عدة صيغ شائعة لتعريف الأنواع في الاستخدام المحددة. فيما يلي بعض الأمثلة:

الأنواع الأساسية

```
let myNumber: number = 123; // number type  
let myBoolean: boolean = true; // boolean type  
let myArray: string[] = ['a', 'b']; // array of strings  
let myTuple: [string, number] = ['a', 123]; // tuple
```

الكائنات والواجهات

```
const x: { name: string; age: number } = { name: 'Simon', age: 7 }
```

أنواع الاتحاد والتقاطع

```
type MyType = string | number; // Union type  
let myUnion: MyType = 'hello'; // Can be a string
```

```

let myUnion: MyType = null; // can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection type
let myCombined: CombinedType = { name: 'John', age: 25 }; // Obj

```

الأنواع الأولية المضمّنة

على عدة أنواع أولية مضمّنة يمكن استخدامها لتعريف TypeScript يحتوي المتغيرات ومعاملات الدوال وأنواع الإرجاع:

- **number**: يمثل القيم العددية، بما في ذلك الأعداد الصحيحة وأعداد الفاصلة العائمة.
- **string**: يمثل البيانات النصية.
- **boolean**: **true** أو **false** يمثل القيم المنطقية التي يمكن أن تكون إما **true** أو **false**.
- **null**: يمثل غياب قيمة.
- **undefined**: يمثل قيمة لم تُسَدَّ أو لم تُعرّف.
- **symbol**: يمثل معرفًا فريدًا. تُستخدم الرموز عادةً بوصفها مفاتيح لخصائص الكائنات.
- **bigint**: يمثل أعدادًا صحيحة بدقة اعتبارية.
- **any**: يمثل نوعًا ديناميكيًا أو مجهولًا. يمكن للمتغيرات من النوع **any** الاحتفاظ بقيم من أي نوع، وهي تتجاوز التحقق من الأنواع.
- **void**: يمثل غياب أي نوع. ويُستخدم عادةً نوع إرجاع للدوال التي لا تُعيد قيمة.
- **never**: يمثل نوعًا للقيم التي لا تحدث مطلقًا. ويُستخدم عادةً نوع إرجاع للدوال التي ترمي خطأ أو تدخل في حلقة لا نهائية.

المضمّنة الشائعة JS كائنات

؛ ويشمل جميع كائنات JavaScript هو مجموعة فائقة من TypeScript المضمّنة شائعة الاستخدام. يمكنك العثور على قائمة موسّعة JavaScript

Mozilla (MDN): بهذه الكائنات في موقع توثيق شبكة مطوري

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

المضمّنة شائعة الاستخدام JavaScript فيما يلي قائمة ببعض كائنات

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

التحميلات الزائدة

تعريف عدة توافيق دوال TypeScript تتيح لك التحميلات الزائدة للدوال في لاسم دالة واحد، مما يتيح لك تعريف دوال يمكن استدعاؤها بطرائق متعددة. إليك مثالاً:

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
```

```
function sayHi(name: unknown): unknown {
  if (typeof name === 'string') {
    return `Hi, ${name}!`;
  } else if (Array.isArray(name)) {
    return name.map(name => `Hi, ${name}!`);
  }
  throw new Error('Invalid value');
}
```

```
sayHi('xx'); // Valid
sayHi(['aa', 'bb']); // Valid
```

class: إليك مثالاً آخر على استخدام التحميلات الزائدة للدوال داخل

```
class Greeter {
  message: string;

  constructor(message: string) {
    this.message = message;
  }

  // overload
  sayHi(name: string): string;
  sayHi(names: string[]): ReadonlyArray<string>;

  // implementation
  sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
      return `${this.message}, ${name}!`;
    } else if (Array.isArray(name)) {
      return name.map(name => `${this.message}, ${name}!`)
    }
    throw new Error('value is invalid');
  }
}

console.log(new Greeter('Hello').sayHi('Simon'));
```

الدمج والتوسيع

يشير الدمج والتوسيع إلى مفهومين مختلفين يتعلقان بالتعامل مع الأنواع والواجهات.

يتيح لك الدمج ضم عدة تصريحات تحمل الاسم نفسه في تعريف واحد، على سبيل المثال، عندما تعرّف واجهة بالاسم نفسه عدة مرات:

```
interface X {  
    a: string;  
}
```

```
interface X {  
    b: number;  
}
```

```
const person: X = {  
    a: 'a',  
    b: 7,  
};
```

يشير التوسيع إلى القدرة على توسيع الأنواع أو الواجهات الموجودة أو الوراثة منها لإنشاء أنواع أو واجهات جديدة. وهو آلية لإضافة خصائص أو أساليب إضافية إلى نوع موجود من دون تعديل تعريفه الأصلي. مثال:

```
interface Animal {  
    name: string;  
    eat(): void;  
}
```

```
interface Bird extends Animal {  
    sing(): void;  
}
```

```
const dog: Bird = {  
    name: 'Bird 1',  
    eat() {
```



```

        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};

```

الاختلافات بين النوع والواجهة

(دمج التصريحات (التوسعة:

تدعم الواجهات دمج التصريحات، ما يعني أنه يمكنك تعريف عدة واجهات في واجهة واحدة تحتوي على TypeScript بالاسم نفسه، وسيدمجها الخصائص والأساليب المجمعّة. وفي المقابل، لا تدعم الأنواع دمج التصريحات. يمكن أن يفيد ذلك عندما تريد إضافة وظائف إضافية أو تخصيص أنواع موجودة من دون تعديل التعريفات الأصلية أو ترقيع الأنواع المفقودة أو غير الصحيحة.

```

interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};

```

توسيع أنواع/واجهات أخرى:

يمكن لكل من الأنواع والواجهات توسيع أنواع/واجهات أخرى، لكن الصياغة لوراثة الخصائص extends مختلفة. مع الواجهات، تستخدم الكلمة المفتاحية والأساليب من واجهات أخرى. ومع ذلك، لا يمكن لواجهة توسيع نوع مركّب مثل نوع الاتحاد.

```

interface A {
  x: string;
  y: number;
}
interface B extends A {
  z: string;
}
const car: B = {
  x: 'x',
  y: 123,
  z: 'z',
};

```

بالنسبة إلى الأنواع، تستخدم المعامل & لدمج عدة أنواع في نوع واحد ((تقاطع).

```

interface A {
  x: string;
  y: number;
}

```

```

type B = A & {
  j: string;
};

```

```

const c: B = {
  x: 'x',
  y: 123,
  j: 'j',
};

```

أنواع الاتحاد والتقاطع:

تكون الأنواع أكثر مرونة عند تعريف أنواع الاتحاد والتقاطع. باستخدام الكلمة يمكنك بسهولة إنشاء أنواع اتحاد باستخدام المعامل | وأنواع type المفتاحية تقاطع باستخدام المعامل &. وعلى الرغم من أن الواجهات يمكنها أيضًا تمثيل أنواع الاتحاد بصورة غير مباشرة، فإنها لا توفر دعمًا مضمّنًا لأنواع التقاطع.

```
type Department = 'dep-x' | 'dep-y'; // Union
```

```
type Person = {  
  name: string;  
  age: number;  
};
```

```
type Employee = {  
  id: number;  
  department: Department;  
};
```

```
type EmployeeInfo = Person & Employee; // Intersection
```

مثال باستخدام الواجهات:

```
interface A {  
  x: 'x';  
}  
interface B {  
  y: 'y';  
}
```

```
type C = A | B; // Union of interfaces
```

الصف

الصياغة الشائعة للصف

لتعريف صف. يمكنك رؤية TypeScript في class تُستخدم الكلمة المفتاحية مثال أدناه:

```
class Person {  
  private name: string;  
  private age: number;  
  constructor(name: string, age: number) {  
    this.name = name;  
    this.age = age;  
  }  
}
```

```

        this.age = age,
    }
    public sayHi(): void {
        console.log(
            `Hello, my name is ${this.name} and I am ${this.age}
        );
    }
}

```

“Person” لتعريف صنف باسم class تُستخدم الكلمة المفتاحية

من age string من النوع name: يحتوي الصنف على خاصيتين خاصيتين number.

age و name يأخذ constructor. يُعرّف المُنشئ باستخدام الكلمة المفتاحية بوصفهما معاملين ويُسندهما إلى الخاصيتين المقابلتين.

يسجّل رسالة ترحيب sayHi باسم public يحتوي الصنف على أسلوب

new يمكنك استخدام الكلمة المفتاحية TypeScript، لإنشاء مثيل لصنف في: متبوعة باسم الصنف، ثم القوسين (). على سبيل المثال:

```

const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I am

```

المُنشئ

المُنشئات هي أساليب خاصة داخل الصنف تُستخدم لتهيئة خصائص الكائن عند إنشاء مثيل للصنف.

```

class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {

```

```

    this.name = name;
    this.age = age;
}

sayHello() {
    console.log(
        `Hello, my name is ${this.name} and I'm ${this.age}
    );
}
}

const john = new Person('Simon', 17);
john.sayHello();

```

يمكن إجراء تحميل زائد لمُنشئ باستخدام الصياغة التالية:

```

type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');

```

يمكن تعريف عدة تحميلات زائدة للمُنشئ، لكن لا يمكن أن TypeScript في يكون لديك سوى تنفيذ واحد يجب أن يكون متوافقًا مع جميع التحميلات الزائدة؛ ويمكن تحقيق ذلك باستخدام معامل اختياري.

```

class Person {
  name: string;
  age: number;

  constructor();
  constructor(name: string);
  constructor(name: string, age: number);
  constructor(name?: string, age?: number) {
    this.name = name ?? 'Unknown';
    this.age = age ?? 0;
  }

  displayInfo() {
    console.log(`Name: ${this.name}, Age: ${this.age}`);
  }
}

```

```

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

```

```

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

```

```

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25

```

الْمُنْشِئَاتُ الْخَاصَةُ وَالْمَحْمِيَّةُ

يمكن رسم الْمُنْشِئَاتُ بِأَنَّهَا خَاصَّةٌ أَوْ مَحْمِيَّةٌ، مِمَّا يَقِيْدُ TypeScript، فِي إِمْكَانِيَّةِ الْوُصُولِ إِلَيْهَا وَاسْتِخْدَامِهَا.

الْمُنْشِئَاتُ الْخَاصَّةُ: لَا يُمْكِنُ اسْتِدْعَاؤُهَا إِلَّا مِنْ دَاخِلِ الصَّنْفِ نَفْسِهِ. وَغَالِبًا مَا تُسْتَخْدَمُ الْمُنْشِئَاتُ الْخَاصَّةُ فِي الْحَالَاتِ الَّتِي تَرِيدُ فِيهَا فَرْضَ نَمَطِ النُّسْخَةِ الْمَفْرَدَةِ أَوْ حَصْرَ إِنْشَاءِ الْمِثْلَاتِ فِي أَسْلُوبِ مَصْنَعِ دَاخِلِ الصَّنْفِ.

الْمُنْشِئَاتُ الْمَحْمِيَّةُ: تَكُونُ الْمُنْشِئَاتُ الْمَحْمِيَّةُ مَفِيدَةً عِنْدَمَا تَرِيدُ إِنْشَاءَ صَنْفٍ أَسَاسِيٍّ يَنْبَغِي عَدَمَ إِنْشَاءِ مِثْلٍ مِنْهُ مَبَاشَرَةً، لَكِنْ يُمْكِنُ تَوْسِيعُهُ بِوَاسِطَةِ

الأصناف الفرعية.

```
class BaseClass {  
    protected constructor() {}  
}
```

```
class DerivedClass extends BaseClass {  
    private value: number;  
  
    constructor(value: number) {  
        super();  
        this.value = value;  
    }  
}
```

```
// Attempting to instantiate the base class directly will result in an error  
// const baseObj = new BaseClass(); // Error: Constructor of class BaseClass is protected
```

```
// Create an instance of the derived class  
const derivedObj = new DerivedClass(10);
```

محددات الوصول

للتحكم في ظهور `public` و `protected` و `private` تُستخدم محددات الوصول أعضاء الصنف وإمكانية الوصول إليها، مثل الخصائص والأساليب، في أصناف TypeScript. وتُعد هذه المحددات أساسية لفرض التغليف ووضع حدود الوصول إلى الحالة الداخلية للصنف وتعديلها.

الوصول إلى عضو الصنف بحيث لا يكون متاحًا إلا داخل `private` يقيّد المحدد الصنف الذي يحتويه.

بالوصول إلى عضو الصنف داخل الصنف الذي `protected` يسمح المحدد. يحتويه وأصنافه المشتقة.

وصولًا غير مقيّد إلى عضو الصنف، مما يسمح بالوصول public يوفر المحدد إليه من أي مكان.

الجلب والضبط

دوال الجلب والضبط هي أساليب خاصة تتيح لك تعريف سلوك مخصص للوصول إلى خصائص الصنف وتعديلها. وهي تمكّنك من تغليف الحالة الداخلية للكائن وتوفير منطق إضافي عند جلب قيم الخصائص أو ضبطها. في TypeScript، تُعرّف دوال الجلب والضبط باستخدام الكلمتين المفتاحيتين `get` و `set` على التوالي. إليك مثالًا:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

الموصلات التلقائية في الأصناف

الإصدار 4.9 دعمًا للموصلات التلقائية، وهي ميزة مرتقبة TypeScript يضيف وهي تشبه خصائص الصنف، لكن يُصرّح عنها باستخدام ECMAScript. في الكلمة المفتاحية “accessor”.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```



```
    }  
}
```

خاصين يعملان على خاصية لا set و get تُحوّل الموصلات التلقائية إلى موصلي يمكن الوصول إليها.

```
class Animal {  
    #__name: string;  
  
    get name() {  
        return this.#__name;  
    }  
    set name(value: string) {  
        this.#__name = value;  
    }  
  
    constructor(name: string) {  
        this.name = name;  
    }  
}
```

this

إلى النسخة المثلثة الحالية this تشير الكلمة المفتاحية، TypeScript، في من الصنف داخل أساليبه أو مُنشئاته. وهي تتيح لك الوصول إلى خصائص الصنف وأساليبه وتعديلها من داخل نطاقه الخاص. وتوفر طريقة للوصول إلى الحالة الداخلية للكائن ومعالجتها داخل أساليبه الخاصة.

```
class Person {  
    private name: string;  
    constructor(name: string) {  
        this.name = name;  
    }  
    public introduce(): void {  
        console.log(`Hello, my name is ${this.name}.`);  
    }  
}
```

```
const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

خصائص المعاملات

تتيح لك خصائص المعاملات التصريح عن خصائص الصنف وتتهيئتها مباشرةً ضمن معاملات المُنشئ، مما يجنبك الشيفرة النمطية المتكررة. على سبيل المثال:

```
class Person {
  constructor(
    private name: string,
    public age: number
  ) {
    // The "private" and "public" keywords in the constructor
    // automatically declare and initialize the corresponding
  }
  public introduce(): void {
    console.log(
      `Hello, my name is ${this.name} and I am ${this.age}
    );
  }
}
const person = new Person('Alice', 25);
person.introduce();
```

الأصناف المجردة

أساسًا للوراثة. وهي توفر TypeScript تُستخدم الأصناف المجردة في طريقة لتعريف خصائص وأساليب مشتركة يمكن أن ترثها الأصناف الفرعية. يفيد ذلك عندما تريد تعريف سلوك مشترك وفرض تنفيذ أساليب معينة على الأصناف الفرعية. وهي توفر طريقة لإنشاء تسلسل هرمي من الأصناف، يوفر فيه الصنف الأساسي المجرد واجهة مشتركة ووظائف مشتركة للأصناف الفرعية.

```

abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.

```

باستخدام الأنواع العامة

تتيح لك الأصناف ذات الأنواع العامة تعريف أصناف قابلة لإعادة الاستخدام يمكنها العمل مع أنواع مختلفة.

```

class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

```

```

    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); // 42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World

```

المزخرفات

توفر المزخرفات آلية لإضافة بيانات وصفية أو تعديل السلوك أو التحقق أو توسيع وظائف العنصر المستهدف. وهي دوال تُنقذ في وقت التشغيل. ويمكن تطبيق عدة مزخرفات على تصريح واحد.

TypeScript المزخرفات ميزات تجريبية، والأمثلة التالية متوافقة فقط مع ES6 الإصدار 5 أو أحدث عند استخدام.

السابقة للإصدار 5، يجب تمكينها TypeScript بالنسبة إلى إصدارات أو tsconfig.json في ملف experimentalDecorators باستخدام الخاصية في سطر الأوامر (لكن المثال التالي experimentalDecorators -- باستخدام (لن يعمل).

تشمل بعض حالات الاستخدام الشائعة للمزخرفات ما يلي:

- مراقبة تغييرات الخصائص.
- مراقبة استدعاءات الأساليب.
- إضافة خصائص أو أساليب إضافية.
- التحقق في وقت التشغيل.
- التسلسل وإلغاء التسلسل تلقائيًا.
- التسجيل.
- التحويل والمصادقة.
- الحماية من الأخطاء.

ملاحظة: لا تسمح مزخرفات الإصدار 5 بزخرفة المعاملات.

أنواع المزخرفات:

مزخرفات الأصناف

تفيد مزخرفات الأصناف في توسيع صنف موجود، مثل إضافة خصائص أو toString أساليب، أو جمع مثيلات صنف. في المثال التالي، نضيف أسلوب يحوّل الصنف إلى تمثيل نصي.

```
type Constructor<T = {}> = new (...args: any[]) => T;
```

```
function toString<Class extends Constructor>(
  Value: Class,
  context: ClassDecoratorContext<Class>
) {
  return class extends Value {
    constructor(...args: any[]) {
      super(...args);
      console.log(JSON.stringify(this));
      console.log(JSON.stringify(context));
    }
  };
}
```

```
@toString
class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  greet() {
    return 'Hello, ' + this.name;
  }
}
```

```
const person = new Person('Simon');
/* Logs:
{"name": "Simon"}
{"kind": "class", "name": "Person"}
*/
```

مزخرف الخاصة

تفيد مزخرفات الخصائص في تعديل سلوك خاصية، مثل تغيير قيم التهيئة. في الشيفرة التالية، لدينا برنامج نصي يضبط خاصية بحيث تكون دائمًا بأحرف كبيرة:

```
function upperCase<T>(
  target: undefined,
  context: ClassFieldDecoratorContext<T, string>
) {
  return function (this: T, value: string) {
    return value.toUpperCase();
  };
}

class MyClass {
  @upperCase
  prop1 = 'hello!';
}
```

```
console.log(new MyClass().prop1); // Logs: HELLO!
```

مزخرف الأسلوب

تتيح لك مزخرفات الأساليب تغيير سلوك الأساليب أو تحسينه. فيما يلي مثال على مُسجِّل بسيط:

```
function log<This, Args extends any[], Return>(
  target: (this: This, ...args: Args) => Return,
  context: ClassMethodDecoratorContext<
    This,
```

```

        (this: This, ...args: Args) => Return
    >
    ) {
        const methodName = String(context.name);

        function replacementMethod(this: This, ...args: Args): Return {
            console.log(`LOG: Entering method '${methodName}'.`);
            const result = target.call(this, ...args);
            console.log(`LOG: Exiting method '${methodName}'.`);
            return result;
        }

        return replacementMethod;
    }

    class MyClass {
        @log
        sayHello() {
            console.log('Hello!');
        }
    }

    new MyClass().sayHello();

```

يُسجَّل ما يلي:

```

LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.

```

مزخرفات دوال الجلب والضبط

تتيح لك مزخرفات دوال الجلب والضبط تغيير سلوك موصلات الصنف أو تحسينه. وهي مفيدة، على سبيل المثال، للتحقق من إسناد الخصائص. إليك مثالاً بسيطاً على مزخرف لدالة جلب:

```

function rande<This> Return extends number>(min: number, max: nu

```

```

        return function (
            target: (this: This) => Return,
            context: ClassGetterDecoratorContext<This, Return>
        ) {
            return function (this: This): Return {
                const value = target.call(this);
                if (value < min || value > max) {
                    throw 'Invalid';
                }
                Object.defineProperty(this, context.name, {
                    value,
                    enumerable: true,
                });
                return value;
            };
        };
    }
}

```

```

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

```

```
const obj = new MyClass(10);
```

```
console.log(obj.getValue); // Valid: 10
```

```
const obj2 = new MyClass(999);
```

```
console.log(obj2.getValue); // Throw: Invalid!
```


البيانات الوصفية للمزخرفات

تُبسَّط البيانات الوصفية للمزخرفات عملية تطبيق البيانات الوصفية واستخدامها بواسطة المزخرفات في أي صنف. ويمكن للمزخرفات الوصول إلى خاصية جديدة للبيانات الوصفية في كائن السياق؛ ويمكن أن تُستخدم مفتاحًا لكل من القيم الأولية والكائنات. يمكن الوصول إلى معلومات البيانات `Symbol.metadata` الوصفية في الصنف عبر

يمكن استخدام البيانات الوصفية لأغراض مختلفة، مثل تصحيح الأخطاء أو التسلسل أو حقن التبعيات باستخدام المزخرفات.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Simple polyfill

type Context =
  | ClassFieldDecoratorContext
  | ClassAccessorDecoratorContext
  | ClassMethodDecoratorContext; // Context contains property

function setMetadata(_target: any, context: Context) {
  // Set the metadata object with a primitive value
  context.metadata[context.name] = true;
}

class MyClass {
  @setMetadata
  a = 123;

  @setMetadata
  accessor b = 'b';

  @setMetadata
  fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Get metadata info
```

```
console.log(JSON.stringify(metadata)); // {"bar":true,"baz":true
```

الوراثة

تشير الوراثة إلى الآلية التي يمكن لصنف من خلالها أن يرث خصائص وأساليب من صنف آخر يُعرف بالصنف الأساسي أو الصنف الفائق. ويمكن للصنف المشتق، الذي يُسمى أيضًا الصنف الابن أو الصنف الفرعي، توسيع وظائف الصنف الأساسي وتخصيصها بإضافة خصائص وأساليب جديدة أو تجاوز الخصائص والأساليب الموجودة.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}
```

```
// Create an instance of the base class
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Create an instance of the derived class
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

الوراثة المتعددة بالمعنى التقليدي، بل يسمح بالوراثة TypeScript لا يدعم واجهات متعددة. يمكن للواجهة TypeScript من صنف أساسي واحد. يدعم تعريف عقد لبنية كائن، ويمكن للصنف تنفيذ عدة واجهات. وهذا يسمح للصنف بوراثة السلوك والبنية من مصادر متعددة.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

كما في TypeScript، في class غالبًا ما يُشار إلى الكلمة المفتاحية ECMAScript 2015 بوصفها سكرًا نحويًا. وقد قُدِّمت في JavaScript،

لتوفير صياغة مألوفة أكثر لإنشاء الكائنات والتعامل معها بأسلوب قائم (ES6) بصفته مجموعة TypeScript، على الأصناف. ومع ذلك، من المهم ملاحظة أن الذي يظل قائمًا JavaScript يُصَرَّف في النهاية إلى JavaScript، فائقة من في جوهره على النماذج الأولية.

الأعضاء الساكنة

على أعضاء ساكنة. للوصول إلى الأعضاء الساكنة في TypeScript يحتوي صنف، يمكنك استخدام اسم الصنف متبوعًا بنقطة، من دون الحاجة إلى إنشاء كائن.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

تهيئة الخصائص

TypeScript: توجد عدة طرائق لتهيئة خصائص صنف في

مباشرة:

في المثال التالي، سُنستخدم هذه القيم الأولية عند إنشاء مثيل للصنف.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

في المُنشئ:

```
class MyClass {
  property1: string;
  property2: number;

  constructor() {
    this.property1 = 'default value';
    this.property2 = 42;
  }
}
```

باستخدام معاملات المُنشئ:

```
class MyClass {
  constructor(
    private property1: string = 'default value',
    public property2: number = 42
  ) {
    // There is no need to assign the values to the properties
  }
  log() {
    console.log(this.property2);
  }
}
const x = new MyClass();
x.log();
```

التحميل الزائد للأساليب

يتيح التحميل الزائد للأساليب للصف أن يحتوي على عدة أساليب بالاسم نفسه، لكن بأنواع معاملات مختلفة أو بعدد مختلف من المعاملات. وهذا يتيح لنا استدعاء أسلوب بطرائق مختلفة بناءً على الوسائط المُمَرَّرة.

```
class MyClass {
  add(a: number, b: number): number; // Overload signature 1
  add(a: string, b: string): string; // Overload signature 2
}
```

```

    add(a: number | string, b: number | string): number | string {
      if (typeof a === 'number' && typeof b === 'number') {
        return a + b;
      }
      if (typeof a === 'string' && typeof b === 'string') {
        return a.concat(b);
      }
      throw new Error('Invalid arguments');
    }
  }
}

```

```

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15

```

الأنواع العامة

تتيح لك الأنواع العامة إنشاء مكوّنات ودوال قابلة لإعادة الاستخدام يمكنها العمل مع أنواع متعددة. وباستخدام الأنواع العامة، يمكنك تحديد معاملات للأنواع والدوال والواجهات، مما يتيح لها العمل على أنواع مختلفة من دون تحديدها صراحةً مسبقًا.

تتيح لك الأنواع العامة جعل الشيفرة أكثر مرونة وقابلية لإعادة الاستخدام.

النوع العام

لتعريف نوع عام، تستخدم قوسين زاويين (<>) لتحديد معاملات النوع، على سبيل المثال:

```

function identity<T>(arg: T): T {
  return arg;
}
const a = identity('x');
const b = identity(123);

```

```
const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

الأصناف العامة

يمكن أيضًا تطبيق الأنواع العامة على الأصناف، وبهذه الطريقة يمكنها العمل مع أنواع متعددة باستخدام معاملات النوع. وهذا مفيد لإنشاء تعريفات أصناف قابلة لإعادة الاستخدام، يمكنها التعامل مع أنواع بيانات مختلفة مع الحفاظ على أمان الأنواع.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}
```

```
const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123
```

```
const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

قيود الأنواع العامة

متبوعةً extends يمكن تقييد المعاملات العامة باستخدام الكلمة المفتاحية بنوع أو واجهة يجب أن يستوفيهها معامل النوع.

ذات نوع صحيح حتى length خاصية T في المثال التالي، يجب أن تكون لدى تكون صالحة:

```
const printLen = <T extends { length: number }>(value: T): void
```

```

        console.log(value.length);
    };

    printLen('Hello'); // 5
    printLen([1, 2, 3]); // 3
    printLen({ length: 10 }); // 10
    printLen(123); // Invalid

```

هي RC إحدى الميزات البارزة للأنواع العامة التي قُدمت في الإصدار 3.4 استدلّال أنواع الدوال ذات الرتبة الأعلى، الذي ينشر وسائط الأنواع العامة

```

declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]

```

الآمن من ناحية الأنواع، وهو pointfree تُسهّل هذه الوظيفة البرمجة بأسلوب أسلوب شائع في البرمجة الوظيفية.

التضييق السياقي للأنواع العامة

TypeScript التضييق السياقي للأنواع العامة هو الآلية التي تتيح لمصرّف تضييق نوع معامل عام استنادًا إلى السياق الذي يُستخدم فيه. ويكون مفيدًا عند العمل مع الأنواع العامة في العبارات الشرطية

```

function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Value is narrowed down to type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {

```



```

        // Value is narrowed down to type 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14

```

الأنواع البنيوية المحمودة

لا يلزم أن تتطابق الكائنات مع نوع محدد ودقيق. على TypeScript، في سبيل المثال، إذا أنشأنا كائنًا يستوفي متطلبات واجهة، فيمكننا استخدام ذلك الكائن في المواضع التي تتطلب هذه الواجهة، حتى لو لم توجد أي صلة صريحة بينهما. مثال:

```

type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Valid

```

مساحات الأسماء

تُستخدم مساحات الأسماء لتنظيم الشيفرة في حاويات TypeScript، في منطقية، ومنع تعارض الأسماء، وتوفير طريقة لتجميع الشيفرة المترابطة معًا.

الوصول إلى مساحة الاسم من خارج export يتيح استخدام الكلمة المفتاحية الوحدات.

```
export namespace MyNamespace {  
  export interface MyInterface1 {  
    prop1: boolean;  
  }  
  export interface MyInterface2 {  
    prop2: string;  
  }  
}  
  
const a: MyNamespace.MyInterface1 = {  
  prop1: true,  
};
```

الرموز

الرموز هي نوع بيانات بدائي يمثل قيمة غير قابلة للتغيير ومضمونة التفرد عالميًا طوال مدة تشغيل البرنامج.

يمكن استخدام الرموز كمفاتيح لخصائص الكائنات، وهي توفر طريقة لإنشاء خصائص غير قابلة للتعداد.

```
const key1: symbol = Symbol('key1');  
const key2: symbol = Symbol('key2');  
  
const obj = {  
  [key1]: 'value 1',  
  [key2]: 'value 2',  
};  
  
console.log(obj[key1]); // value 1  
console.log(obj[key2]); // value 2
```

WeakMap و WeakSet أصبح استخدام الرموز كمفاتيح مسموحًا الآن في

توجيهات الشروط المائلة الثلاث

توجيهات الشروط المائلة الثلاث هي تعليقات خاصة تقدّم إلى المصرّف تعليمات حول كيفية معالجة ملف. تبدأ هذه التوجيهات بثلاث شروط مائلة ولا تؤثر في السلوك، TypeScript، متتالية (///)، وتوضع عادةً في أعلى ملف وقت التشغيل.

تُستخدم توجيهات الشروط المائلة الثلاث للإشارة إلى التبعيات الخارجية، وتحديد سلوك تحميل الوحدات، وتمكين بعض ميزات المصرّف أو تعطيلها، وغير ذلك. فيما يلي بعض الأمثلة:

الإشارة إلى ملف تصريح:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

تحديد تنسيق الوحدة:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

تمكين خيارات المصرّف، وفي المثال التالي الوضع الصارم:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

معالجة الأنواع

إنشاء أنواع من أنواع أخرى

يمكن إنشاء أنواع جديدة من خلال تركيب الأنواع الموجودة أو معالجتها أو تحويلها.

أنواع التقاطع (&):

تتيح لك دمج أنواع متعددة في نوع واحد:

```

type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };

```

(|): أنواع الاتحاد

تتيح لك تعريف نوع يمكن أن يكون واحدًا من عدة أنواع

```

type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;

```

الأنواع المعيّنة

تتيح لك تحويل خصائص نوع موجود لإنشاء نوع جديد

```

type Mutable<T> = {
  readonly [P in keyof T]: T[P];
};
type Person = {
  name: string;
  age: number;
};
type ImmutablePerson = Mutable<Person>; // Properties become read-only

```

الأنواع الشرطية

تتيح لك إنشاء أنواع استنادًا إلى بعض الشروط

```

type ExtractParam<T> = T extends (param: infer P) => any ? P : never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string

```

أنواع الوصول المفهرس

يمكن الوصول إلى أنواع الخصائص الموجودة داخل نوع TypeScript، في `Type[Key]`، آخر ومعالجتها باستخدام فهرس.

```
type Person = {  
  name: string;  
  age: number;  
};
```

```
type AgeType = Person['age']; // number
```

```
type MyTuple = [string, number, boolean];  
type MyType = MyTuple[2]; // boolean
```

أنواع الأدوات المساعدة

يمكن استخدام عدة أنواع أدوات مساعدة مضمّنة لمعالجة الأنواع، وفيما يلي قائمة بأكثرها استخدامًا:

Awaited<T>

بصورة متكررة Promise ينشئ نوعًا يفك تغليف أنواع.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

على أنها اختيارية T ينشئ نوعًا تُعَيَّن فيه جميع خصائص.

```
type Person = {  
  name: string;  
  age: number;  
};
```

```
type A = Partial<Person>; // { name?: string | undefined; age?:
```

Required<T>

على أنها مطلوبة T ينشئ نوعًا تُعَيَّن فيه جميع خصائص

```
type Person = {  
  name?: string;  
  age?: number;  
};  
  
type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

على أنها للقراءة فقط T ينشئ نوعًا تُعَيَّن فيه جميع خصائص

```
type Person = {  
  name: string;  
  age: number;  
};  
  
type A = Readonly<Person>;  
  
const a: A = { name: 'Simon', age: 17 };  
a.name = 'John'; // Invalid
```

Record<K, T>

T من النوع K ينشئ نوعًا يحتوي على مجموعة من الخصائص

```
type Product = {  
  name: string;  
  price: number;  
};  
  
const products: Record<string, Product> = {  
  apple: { name: 'Apple', price: 0.5 },  
};
```

```

    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }

```

Pick<T, K>

T من K ينشئ نوعًا بانتقاء الخصائص المحددة

```

type Product = {
  name: string;
  price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }

```

Omit<T, K>

T من K ينشئ نوعًا بحذف الخصائص المحددة

```

type Product = {
  name: string;
  price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }

```

Exclude<T, U>

T من U ينشئ نوعًا باستبعاد جميع قيم النوع

```

type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b

```

Extract<T, U>

T. من U ينشئ نوعًا باستخراج جميع قيم النوع

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a / c
```

NonNullable<T>

T. من null و undefined ينشئ نوعًا باستبعاد

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

T. يستخرج أنواع معاملات نوع الدالة

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

T. يستخرج أنواع معاملات نوع دالة المُنشئ

```
class Person {
  constructor(
    public name: string,
    public age: number
  ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof Person>;
const params: PersonConstructorParams = ['John', 30];

const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```


ReturnType<T>

T. يستخرج نوع الإرجاع لنوع الدالة

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

T. يستخرج نوع الممثل لنوع الصنف

```
class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  sayHello() {
    console.log(`Hello, my name is ${this.name}!`);
  }
}
```

```
type PersonInstance = InstanceType<typeof Person>;
```

```
const person: PersonInstance = new Person('John');
```

```
person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

T. من نوع الدالة 'this' يستخرج نوع معامل

```
interface Person {
  name: string;
  greet(this: Person): void;
}
```

```

type PersonThisType = ThisParameterType<Person['greet']>; // PersonThisType

```

OmitThisParameter<T>

T من نوع الدالة 'this' يزيل معامل.

```

function capitalize(this: String) {
    return this[0].toUpperCase() + this.substring(1).toLowerCase();
}

```

```

type CapitalizeType = OmitThisParameter<typeof capitalize>; // CapitalizeType

```

ThisType<T>

سياقي this يعمل كعلامة لنوع.

```

type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // Valid as "log" is a part of Logger
        this.update(); // Invalid
    },
};

```

Uppercase<T>

إلى أحرف كبيرة T يحوّل اسم نوع الإدخال.

```

type MyType = Uppercase<'abc'>; // "ABC"

```

Lowercase<T>

إلى أحرف صغيرة T يحوّل اسم نوع الإدخال.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

إلى حرف كبير T يحوّل الحرف الأول من اسم نوع الإدخال.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

إلى حرف صغير T يحوّل الحرف الأول من اسم نوع الإدخال.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer هو نوع أداة مساعدة مصمم لمنع الاستدلال التلقائي للأنواع ضمن NoInfer. نطاق دالة عامة.

مثال:

```
// Automatic inference of types within the scope of a generic function
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" | "c"),
```

باستخدام NoInfer:

```
// Example function that uses NoInfer to prevent type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
```

```
    return x.concat(y);  
}
```

```
const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of type
```

مواضيع أخرى

الأخطاء ومعالجة الاستثناءات

JavaScript التقاط الأخطاء ومعالجتها باستخدام آليات TypeScript تتيح لك القياسية لمعالجة الأخطاء:

كتل Try-Catch-Finally:

```
try {  
    // Code that might throw an error  
} catch (error) {  
    // Handle the error  
} finally {  
    // Code that always executes, finally is optional  
}
```

يمكنك أيضًا معالجة أنواع مختلفة من الأخطاء:

```
try {  
    // Code that might throw different types of errors  
} catch (error) {  
    if (error instanceof TypeError) {  
        // Handle TypeError  
    } else if (error instanceof RangeError) {  
        // Handle RangeError  
    } else {  
        // Handle other errors  
    }  
}
```

أنواع الأخطاء المخصصة:

class Error: يمكن تحديد أخطاء أكثر تخصيصًا من خلال توسيع

```
class CustomError extends Error {  
    constructor(message: string) {  
        super(message);  
        this.name = 'CustomError';  
    }  
}
```

```
throw new CustomError('This is a custom error.');
```

أصناف المزج

تتيح لك أصناف المزج دمج سلوك من أصناف متعددة وتركيبه في صنف واحد. وهي توفر طريقة لإعادة استخدام الوظائف وتوسيعها دون الحاجة إلى سلاسل وراثية عميقة.

```
abstract class Identifiable {  
    name: string = '';  
    logId() {  
        console.log('id:', this.name);  
    }  
}  
  
abstract class Selectable {  
    selected: boolean = false;  
    select() {  
        this.selected = true;  
        console.log('Select');  
    }  
    deselect() {  
        this.selected = false;  
        console.log('Deselect');  
    }  
}  
  
class MyClass {  
    constructor() {}  
}
```

```
// Extend MyClass to include the behavior of Identifiable and Se  
interface MyClass extends Identifiable, Selectable {}
```

```
// Function to apply mixins to a class  
function applyMixins(source: any, baseCtors: any[]) {  
    baseCtors.forEach(baseCtor => {  
        Object.getOwnPropertyNames(baseCtor.prototype).forEach(r  
            let descriptor = Object.getOwnPropertyDescriptor(  
                baseCtor.prototype,  
                name  
            );  
            if (descriptor) {  
                Object.defineProperty(source.prototype, name, de  
            }  
        });  
    });  
}
```

```
// Apply the mixins to MyClass  
applyMixins(MyClass, [Identifiable, Selectable]);  
let o = new MyClass();  
o.name = 'abc';  
o.logId();  
o.select();
```

مميزات اللغة غير المتزامنة

فهي تتضمن ميزات JavaScript، مجموعة شاملة من TypeScript لأن المضمّنة للغة غير المتزامنة مثل JavaScript:

الوعد:

الوعد هي طريقة لمعالجة العمليات غير المتزامنة ونتائجها باستخدام توابع لمعالجة حالتي النجاح والخطأ `catch()` و `then()` مثل.

لمعرفة المزيد: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

صياغة تبدو أكثر تزامنًا عند العمل Async/await توفر الكلمتان المفتاحيتان لتعريف دالة غير متزامنة، async مع الوعود. تُستخدم الكلمة المفتاحية داخل دالة غير متزامنة لإيقاف التنفيذ مؤقتًا await وتُستخدم الكلمة المفتاحية بالقبول أو الرفض Promise إلى أن تتم تسوية.

لمعرفة المزيد: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await>

TypeScript التالية مدعومة جيدًا في API واجهات

واجهة Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: <https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker>

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

المكثرات والمولدات

المكثرات والمولدات دعمًا جيدًا TypeScript تدعم.

المكرّرات هي كائنات تطبّق بروتوكول التكرار، وتوفّر طريقة للوصول إلى عناصر مجموعة أو تسلسل واحدًا تلو الآخر. وهي بنية تحتوي على مؤشر إلى يعيد القيمة التالية في `next()` العنصر التالي في عملية التكرار. ولديها تابع التسلسل إلى جانب قيمة منطقية تشير إلى ما إذا كان التسلسل قد أصبح `done`.

```
class NumberIterator implements Iterable<number> {
  private current: number;

  constructor(
    private start: number,
    private end: number
  ) {
    this.current = start;
  }

  public next(): IteratorResult<number> {
    if (this.current <= this.end) {
      const value = this.current;
      this.current++;
      return { value, done: false };
    } else {
      return { value: undefined, done: true };
    }
  }

  [Symbol.iterator](): Iterator<number> {
    return this;
  }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
  console.log(num);
}
```


التي تبسّط `function*` المولّدات هي دوال خاصة تُعرّف باستخدام صياغة لتعريف تسلسل `yield` إنشاء المكرّرات. وهي تستخدم الكلمة المفتاحية `yield` لتعريف تسلسل `yield` إنشاء المكرّرات، وهي مفيدة بوجه خاص عند العمل مع تسلسلات كبيرة أو غير منتهية.

تسهّل المولّدات إنشاء المكرّرات، وهي مفيدة بوجه خاص عند العمل مع تسلسلات كبيرة أو غير منتهية.

مثال:

```
function* numberGenerator(start: number, end: number): Generator {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}
```

```
const generator = numberGenerator(1, 5);
```

```
for (const num of generator) {
  console.log(num);
}
```

أيضًا المكرّرات غير المتزامنة والمولّدات غير المتزامنة TypeScript تدعم.

لمعرفة المزيد:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

TsDocs و JSDoc مرجع

على TypeScript يمكن مساعدة JavaScript عند العمل مع قاعدة شيفرة مع شروح توضيحية إضافية JSDoc استدلال النوع الصحيح باستخدام تعليقات

لتوفير معلومات النوع.

مثال:

```
/**
 * Computes the power of a given number
 * @constructor
 * @param {number} base - The base value of the expression
 * @param {number} exponent - The exponent value of the expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number).
```

تتوفر الوثائق الكاملة على هذا الرابط:

<https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html>

JSDoc من صياغة d.ts. بدءًا من الإصدار 3.7، يمكن إنشاء تعريفات الأنواع في JavaScript. يمكن العثور على مزيد من المعلومات هنا:

<https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html>

@types

هي اصطلاحات خاصة لتسمية الحزم، @types الحزم التابعة لمنظمة أو وحداتها الموجودة. JavaScript تُستخدم لتوفير تعريفات الأنواع لمكتبات: على سبيل المثال، سيؤدي استخدام

```
npm install --save-dev @types/lodash
```

في مشروعك الحالي lodash إلى تثبيت تعريفات الأنواع الخاصة بـ

يرجى إرسال طلب ،@types للمساهمة في تعريفات الأنواع الخاصة بحزمة <https://github.com/DefinitelyTyped/DefinitelyTyped>. سحب إلى

JSX

يتيح لك كتابة JavaScript هو امتداد لصياغة لغة (JavaScript XML) JSX ويُستخدم TypeScript أو JavaScript داخل ملفات HTML شيفرة شبيهة بـ HTML. لتعريف بنية React عادةً في

من خلال توفير فحص الأنواع والتحليل JSX إمكانات TypeScript توسّع الساكن.

tsconfig.json في ملف jsx تحتاج إلى تعيين خيار المصّرّف ،JSX لاستخدام :وفيما يلي خياران شائعان للإعداد

- دون تغيير. يطلب هذا الخيار JSX مع إبقاء jsx. يُنتج ملفات “preserve” كما هي وعدم تحويلها أثناء JSX الحفاظ على صياغة TypeScript من عملية التصريف. يمكنك استخدام هذا الخيار إذا كانت لديك أداة منفصلة، تتولى التحويل، مثل Babel.
- سيُستخدم TypeScript المضمّن في JSX يمكن تحويل “react” :React.createElement.

:تتوفر جميع الخيارات هنا

<https://www.typescriptlang.org/tsconfig#jsx>

ES6 وحدات

والعديد من الإصدارات (ECMAScript 2015) TypeScript ES6 تدعم مثل الدوال السهمية، ES6 اللاحقة. ويعني ذلك أنه يمكنك استخدام صياغة وقوالب النصوص الحرفية، والأصناف، والوحدات، والتفكيك، وغير ذلك.

في target في مشروعك، يمكنك تحديد الخاصية ES6 لتمكين ميزات tsconfig.json.

مثال على إعداد:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

ES7 عامل الأس في

يحسب عامل الأس (**) القيمة الناتجة من رفع المُعامل الأول إلى قوة لكن مع قدرة إضافية، `Math.pow()` المُعامل الثاني. ويعمل بصورة مشابهة لـ هذا العامل بالكامل عند TypeScript كُمعاملات. تدعم BigInts على قبول أو إصدار أحدث es2016 إلى `tsconfig.json` في ملف `target` تعيين.

```
console.log(2 ** (2 ** 2)); // 16
```

عبارة for-await-of

وتتيح لك التكرار، TypeScript مدعومة بالكامل في JavaScript هذه ميزة es2018 عبر الكائنات القابلة للتكرار غير المتزامنة مع الإصدار المستهدف.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
  yield Promise.resolve(1);
  yield Promise.resolve(2);
  yield Promise.resolve(3);
}
```

```
(async () => {
  for await (const num of asyncNumbers()) {
    console.log(num);
  }
})()
```

```
    }  
  })();
```

new.target الخاصية الوصفية

وهي TypeScript، في new.target يمكنك استخدام الخاصية الوصفية وتيح new.target من تحديد ما إذا استدعيت دالة أو مُنشئ باستخدام العامل لك اكتشاف ما إذا أنشئ كائن نتيجة استدعاء مُنشئ.

```
class Parent {  
  constructor() {  
    console.log(new.target); // Logs the constructor function  
  }  
}  
  
class Child extends Parent {  
  constructor() {  
    super();  
  }  
}  
  
const parentX = new Parent(); // [Function: Parent]  
const child = new Child(); // [Function: Child]
```

تعبيرات الاستيراد الديناميكي

يمكن تحميل الوحدات شرطياً أو تحميلها تحميلاً كسوّلاً عند الطلب باستخدام TypeScript للاستيراد الديناميكي، وهو مدعوم في ECMAScript مقترح

كما يلي TypeScript تكون صياغة تعبيرات الاستيراد الديناميكي في

```
async function renderWidget() {  
  const container = document.getElementById('widget');  
  if (container !== null) {  
    const widget = await import('./widget'); // Dynamic import  
    widget.render(container);  
  }  
}
```

```
}
```

```
renderWidget();
```

“tsc –watch”

مع إمكانية إعادة `--watch` مع المعامل TypeScript يُشغّل هذا الأمر مصرّف تلقائيًا كلما عُدلت TypeScript تصريف ملفات.

```
tsc --watch
```

الإصدار 4.9، تعتمد مراقبة الملفات أساسًا على أحداث TypeScript بدءًا من نظام الملفات، وتلجأ تلقائيًا إلى الاستقصاء إذا تعذر إنشاء مراقب قائم على الأحداث.

عامل التوكيد بعدم انعدام القيمة

عامل التوكيد بعدم انعدام القيمة (اللاحقة !)، الذي يُسمى أيضًا توكيدات تتيح لك تأكيد أن متغيرًا أو خاصية TypeScript الإسناد المحدد، هو ميزة في الساكن TypeScript حتى إذا أشار تحليل `undefined` أو `null` ليست للأنواع إلى احتمال أن تكون كذلك. تتيح هذه الميزة إزالة أي فحص صريح

```
type Person = {  
  name: string;  
};
```

```
const printName = (person?: Person) => {  
  console.log(`Name is ${person!.name}`);  
};
```

التصريحات ذات القيم الافتراضية

تُستخدم التصريحات ذات القيم الافتراضية عندما يُسَدّ إلى متغير أو معامل قيمة افتراضية. ويعني ذلك أنه إذا لم تُقدّم قيمة لذلك المتغير أو المعامل،

فستُستخدم القيمة الافتراضية بدلاً منها.

```
function greet(name: string = 'Anonymous'): void {  
    console.log(`Hello, ${name}!`);  
}  
greet(); // Hello, Anonymous!  
greet('John'); // Hello, John!
```

التسلسل الاختياري

يعمل عامل التسلسل الاختياري ?. مثل عامل النقطة العادي (.) للوصول إلى بإنهاء undefined أو null الخصائص أو التوابع. لكنه يتعامل بسلاسة مع قيم بدلاً من طرح خطأ undefined التعبير وإعادة

```
type Person = {  
    name: string;  
    age?: number;  
    address?: {  
        street?: string;  
        city?: string;  
    };  
};  
  
const person: Person = {  
    name: 'John',  
};  
  
console.log(person.address?.city); // undefined
```

عامل الدمج المنعدم

يعيد عامل الدمج المنعدم ?? قيمة الطرف الأيمن إذا كان الطرف الأيسر null أو undefined قيمة الطرف الأيسر. وإلا فيعيد قيمة الطرف الأيسر.

```
const foo = null ?? 'foo';  
console.log(foo); // foo
```

```
const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

أنواع قوالب النصوص الحرفية

تتيح لك أنواع قوالب النصوص الحرفية معالجة قيم السلاسل النصية على مستوى النوع وإنشاء أنواع سلاسل نصية جديدة استنادًا إلى الأنواع الموجودة. وهي مفيدة لإنشاء أنواع أكثر تعبيرًا ودقة من العمليات القائمة على السلاسل النصية.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id"
```

التحميل الزائد للدوال

يتيح لك التحميل الزائد للدوال تعريف عدة توافيق للدالة نفسها، لكل منها أنواع معاملات وأنواع إرجاع مختلفة. عند استدعاء دالة محملة زائدًا، الوسائط المقدّمة لتحديد توقيع الدالة الصحيح TypeScript تستخدم:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
  if (typeof person === 'string') {
    return `Hi ${person}!`;
  } else if (Array.isArray(person)) {
    return person.map(name => `Hi, ${name}!`);
  }
  throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```


الأنواع العودية

النوع العودي هو نوع يمكنه الإشارة إلى نفسه. وهذا مفيد لتعريف بُنى بيانات لها بنية هرمية أو عودية (قد يكون تداخلها غير منتهٍ)، مثل القوائم المرتبطة، والأشجار، والرسوم البيانية.

```
type ListNode<T> = {  
  data: T;  
  next: ListNode<T> | undefined;  
};
```

الأنواع الشرطية العودية

يمكن تعريف علاقات أنواع معقدة باستخدام المنطق والعودية في TypeScript. لنوضح الأمر بعبارات بسيطة:

تتيح لك الأنواع الشرطية تعريف أنواع استنادًا إلى شروط منطقية:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number'  
type A = CheckNumber<123>; // 'Number'  
type B = CheckNumber<'abc'>; // 'Not a number'
```

تعني العودية تعريف نوع يشير إلى نفسه داخل تعريفه:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };  
  
const data: Json = {  
  prop1: true,  
  prop2: 'prop2',  
  prop3: {  
    prop4: [],  
  },  
};
```

تجمع الأنواع الشرطية العودية بين المنطق الشرطي والعودية. وهذا يعني أن تعريف نوع يمكن أن يعتمد على نفسه من خلال منطق شرطي، ما ينشئ علاقات أنواع معقدة ومرنة.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;
```

```
type NestedArray = [1, [2, [3, 4], 5], 6];  
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 6
```

Node في ECMAScript دعم وحدات

بدءًا من الإصدار 15.3.0، وتدعم ECMAScript دعم وحدات Node.js أضاف منذ الإصدار 4.7. يمكن في Node.js ECMAScript وحدات TypeScript في ملف nodenext بالقيمة module تمكين هذا الدعم باستخدام الخاصية tsconfig.json. إليك مثالًا:

```
{  
  "compilerOptions": {  
    "module": "nodenext",  
    "outDir": "./lib",  
    "declaration": true  
  }  
}
```

لوحدة ES و cjs. ES لوحدة mjs: امتدادين لملفات الوحدات Node.js يدعم ES لوحدة mts. هما TypeScript والامتدادان المكافئان في CommonJS. هذه الملفات TypeScript عندما يحوّل مصرّف CommonJS لوحدة cts. و cjs. فإنه ينشئ ملفات JavaScript، إلى mjs و cjs.

في مشروعك، فيمكنك تعيين الخاصية ES إذا كنت تريد استخدام وحدات Node.js يطلب ذلك من package.json في ملف "module" إلى type ES. التعامل مع المشروع على أنه مشروع وحدات

أيضًا تصريحات الأنواع في ملفات TypeScript بالإضافة إلى ذلك، تدعم توقُّر ملفات التصريحات هذه معلومات الأنواع للمكتبات أو الوحدات .d.ts. ما يتيح للمطورين الآخرين استخدامها مع ميزات TypeScript المكتوبة بلغة TypeScript. فحص الأنواع والإكمال التلقائي في TypeScript.

دوال التوكيد

دوال التوكيد هي دوال تشير إلى التحقق من شرط محدد، TypeScript في استنادًا إلى قيمة الإرجاع الخاصة بها. في أبسط صورها، تفحص دالة التوكيد false مُسندًا مقدَّمًا وتطرح خطأ عندما تكون نتيجة المُسند

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

أو يمكن التصريح عنها كتعبير دالة:

```
type AssertIsNumber = (value: unknown) => asserts value is number
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

تتشابه دوال التوكيد مع حراس الأنواع. كان الغرض من تقديم حراس الأنواع أصلًا إجراء عمليات تحقق وقت التشغيل وضمان نوع قيمة ضمن نطاق محدد. وبوجه خاص، حارس النوع هو دالة تقيِّم مُسند نوع وتعيد قيمة منطقية تشير وهذا يختلف قليلًا عن دوال التوكيد، إذ false أو true إلى ما إذا كان المُسند. عندما لا يتحقق المُسند false يكون القصد فيها طرح خطأ بدلًا من إعادة.

مثال على حارس النوع:

```
const isNumber = (value: unknown): value is number => typeof val
```

أنواع الصفوف متغيرة الطول

أنواع الصفوف متغيرة الطول هي ميزة أُضيفت في الإصدار 4.0 من TypeScript، لذا لنبدأ بمراجعة مفهوم الصف

نوع الصف هو مصفوفة ذات طول محدد، ويكون نوع كل عنصر فيها معروفاً

```
type Student = [string, number];  
const [name, age]: Student = ['Simone', 20];
```

يعني مصطلح “متغير الطول” عدد معاملات غير محدد (قبول عدد متغير من الوسائط).

الصف متغير الطول هو نوع صف يمتلك جميع الخصائص السابقة، لكن شكله الدقيق لم يُحدّد بعد

```
type Bar<T extends unknown[]> = [boolean, ...T, number];
```

```
type A = Bar<[boolean]>; // [boolean, boolean, number]  
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]  
type C = Bar<[]>; // [boolean, number]
```

العامة التي T يمكننا أن نرى في الشيفرة السابقة أن شكل الصف تحدده جري تمريرها.

يمكن للصفوف متغيرة الطول قبول عدة أنواع عامة، مما يجعلها شديدة المرونة

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];
```

```
type A = Bar<[number], [string]>; // [number, boolean, string]  
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

مع الصفوف الجديدة متغيرة الطول، يمكننا استخدام ما يلي

- يمكن الآن أن تكون عوامل النشر في صياغة نوع الصف عامة، ولذلك يمكننا تمثيل العمليات ذات الرتبة الأعلى على الصفوف والمصفوفات حتى عندما لا نعرف الأنواع الفعلية التي نتعامل معها.
- يمكن أن تظهر عناصر الباقي في أي موضع داخل الصف.

مثال:

```
type Items = readonly unknown[];
```

```
function concat<T extends Items, U extends Items>(
  arr1: T,
  arr2: U
): [...T, ...U] {
  return [...arr1, ...arr2];
}
```

```
concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

الأنواع المٌغلّفة

تشير الأنواع المٌغلّفة إلى الكائنات المٌغلّفة المستخدمة لتمثيل الأنواع الأولية في صورة كائنات. توفر هذه الكائنات المٌغلّفة وظائف وأساليب إضافية غير متاحة مباشرةً على القيم الأولية.

على قيمة أولية من النوع `normalize` أو `charAt` عندما تصل إلى أسلوب مثل `string`، وتستدعي الأسلوب، ثم `String` داخل كائن `JavaScript` تغلفها، تتخلص من الكائن.

توضيح:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
  console.log(this, typeof this);
  return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

هذا الاختلاف بتوفير أنواع منفصلة للأنواع الأولية وأغلفة TypeScript تمثل الكائنات المقابلة لها:

- `string => String`
- `number => Number`
- `boolean => Boolean`
- `symbol => Symbol`
- `bigint => BigInt`

لا تكون الأنواع المُغلَّفة مطلوبة عادةً. تجنب استخدامها واستعمل الأنواع `String` بدلاً من `string` الأولية بدلاً منها، مثل

TypeScript التغير والتغير العكسي في

يصف التغير والتغير العكسي كيفية تصرف علاقات الأنواع داخل الأنواع العامة.

في TypeScript:

- المصفوفات **متغيرة**، لكن هذا ليس آتياً تماماً من ناحية الأنواع.
- أنواع معاملات الدوال تكون:
 - `strictFunctionTypes` **متغيرة عكسياً** عند تفعيل
 - **متغيرة في الاتجاهين** بخلاف ذلك

نوعاً فرعياً من النوع A يعني التغير أن العلاقة تبقى محفوظة: إذا كان النوع يظهر هذا في TypeScript، $F < B >$ يكون أيضاً نوعاً فرعياً من $F < A >$ فإن B ، عادةً في أنواع الإرجاع وفي المصفوفات (مع أن تغير المصفوفات ليس آتياً تماماً من ناحية الأنواع).

نوعاً فرعياً من النوع A يعني التغير العكسي أن العلاقة تنعكس: إذا كان النوع يُفترض أن تكون في TypeScript، $F < A >$ يكون نوعاً فرعياً من $F < B >$ فإن B ، أنواع معاملات الدوال متغيرة عكسياً، مما يعني إمكان استخدام دالة تقبل نوعاً أوسع حيث يُتوقع نوع أضيق.

غالبًا بالتغاير في الاتجاهين لمعاملات الدوال TypeScript لكن عمليًا، تسمح مفعلاً، مما يعني إمكان قبول الاتجاهين strictFunctionTypes (ما لم يكن حتى عندما لا يكون ذلك آمنًا تمامًا من ناحية الأنواع

مثال: تخيل مساحة لجميع الحيوانات ومساحة منفصلة للكلاب فقط.

- **التغاير:**

يمكنك استخدام “مساحة كلاب” حيث يُتوقع وجود “مساحة حيوانات”، لأن جميع الكلاب حيوانات.

لكن لا يمكنك استخدام “مساحة حيوانات” حيث يُتوقع وجود “مساحة كلاب”، لأنها قد تحتوي على حيوانات ليست كلابًا.

- **(التغاير العكسي) (فكر من منظور الدوال:**

إذا كان لديك شيء يمكنه التعامل مع أي حيوان، فيمكنك استخدامه حيث يُتوقع شيء يتعامل مع الكلاب فقط. لكن العكس غير صحيح.

مثال على التغاير:

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
}

class Dog extends Animal {
  breed: string;
  constructor(name: string, breed: string) {
    super(name);
    this.breed = breed;
  }
}

let animals: Animal[] = [];
let dogs: Dog[] = [];
```

```
// Arrays are covariant in TypeScript (but not type-safe)  
animals = dogs; // allowed  
dogs = animals; // error
```

مثال على التغير العكسي:

```
class Animal {  
    name: string;  
    constructor(name: string) {  
        this.name = name;  
    }  
}  
  
class Dog extends Animal {  
    breed: string;  
    constructor(name: string, breed: string) {  
        super(name);  
        this.breed = breed;  
    }  
}  
  
type Feed<T> = (animal: T) => void;  
  
let feedAnimal: Feed<Animal> = animal => {  
    console.log(animal.name);  
};  
  
let feedDog: Feed<Dog> = dog => {  
    console.log(dog.breed);  
};  
  
// Intended contravariance:  
feedDog = feedAnimal; // safe  
  
// This depends on compiler settings:  
feedAnimal = feedDog; // error only with strictFunctionTypes
```


تعليقات التغير الاختيارية لمعاملات الأنواع

in و out يمكننا استخدام الكلمتين المفتاحيتين، TypeScript 4.7.0 بدءًا من لتحديد تعليقات التغير.

out: للتغير، استخدم الكلمة المفتاحية

```
type AnimalCallback<out T> = () => T; // T is Covariant here
```

in: وللتغير العكسي، استخدم الكلمة المفتاحية

```
type AnimalCallback<in T> = (value: T) => void; // T is Contravariant
```

تواقيع الفهرس ذات أنماط السلاسل القالبية

تتيح لنا تواقيع الفهرس ذات أنماط السلاسل القالبية تعريف تواقيع فهرس مرنة باستخدام أنماط السلاسل القالبية. تمكّننا هذه الميزة من إنشاء كائنات يمكن فهرستها باستخدام أنماط محددة من المفاتيح النصية، مما يوفر مزيدًا من التحكم والدقة عند الوصول إلى الخصائص ومعالجتها.

بدءًا من الإصدار 4.4، بتواقيع الفهرس للرموز وأنماط، TypeScript تسمح السلاسل القالبية.

```
const uniqueSymbol = Symbol('description');
```

```
type MyKeys = `key-${string}`;
```

```
type MyObject = {  
  [uniqueSymbol]: string;  
  [key: MyKeys]: number;  
};
```

```
const obj: MyObject = {  
  [uniqueSymbol]: 'Unique symbol key',  
  'key-a': 123,  
};
```

```

    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456

```

المعامل satisfies

التحقق مما إذا كان نوع معين يفي بواجهة أو satisfies يتيح لك المعامل شرط محدد. بعبارة أخرى، يضمن أن النوع يمتلك جميع الخصائص والأساليب المطلوبة لواجهة معينة. وهو طريقة للتأكد من أن متغيرًا يطابق تعريف نوع. إليك مثالًا:

```

type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// In the following lines, TypeScript won't be able to infer properly
user.attributes?.map(console.log); // Property 'map' does not exist on type 'string[] | undefined'
user.nickName; // string | string[] | undefined

// Type assertion using `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Here too, TypeScript won't be able to infer properly

```

```
user2.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[] | undefined'
user2.nickName; // string | string[] | undefined
```

```
// Using the `satisfies` operator we can properly infer the type
const user3 = {
  name: 'Simon',
  nickName: undefined,
  attributes: ['dev', 'admin'],
} satisfies User;
```

```
user3.attributes?.map(console.log); // TypeScript infers correctly: undefined
user3.nickName; // TypeScript infers correctly: undefined
```

عمليات الاستيراد والتصدير الخاصة بالأنواع فقط

تتيح لك عمليات الاستيراد والتصدير الخاصة بالأنواع فقط استيراد الأنواع أو تصديرها دون استيراد أو تصدير القيم أو الدوال المرتبطة بتلك الأنواع. قد يفيد ذلك في تقليل حجم حزمك.

لاستخدام عمليات الاستيراد الخاصة بالأنواع فقط، يمكنك استخدام الكلمة `import type`.

(.ts باستخدام امتدادات ملفات التعريف والتنفيذ معًا TypeScript تسمح في عمليات الاستيراد الخاصة بالأنواع فقط، بصرف النظر (.tsx و .cts و .mts و `allowImportingTsExtensions` عن إعدادات

على سبيل المثال:

```
import type { House } from './house.ts';
```

الصيغ التالية مدعومة:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
```

```
export type { T };
export type { T } from './mod';
```

والإدارة الصريحة للموارد using تصريح

`const`، هو ارتباط غير قابل للتغيير ضمن نطاق كتلة، ويشبه `using` تصريح `وَيُستَخدم لإدارة الموارد القابلة للتخلص منها`. عند تهيئته بقيمة، `يُسجَل الخاص بتلك القيمة ثم يُنفَّذ عند الخروج من نطاق Symbol.dispose أسلوب الكتلة المحيطة`.

وهي مفيدة لتنفيذ ECMAScript، يعتمد ذلك على ميزة إدارة الموارد في مهام التنظيف الأساسية بعد إنشاء الكائن، مثل إغلاق الاتصالات وحذف الملفات وتحرير الذاكرة.

:ملاحظات

- تفتقر معظم TypeScript، نظرًا إلى إضافتها حديثًا في الإصدار 5.2 من: بيئات التشغيل إلى الدعم الأصلي. ستحتاج إلى إضافات توافق من أجل `DisposableStack` و `Symbol.asyncDispose` و `Symbol.dispose` و `SuppressedError` و `AsyncDisposableStack`.
- على النحو التالي `tsconfig.json` بالإضافة إلى ذلك، ستحتاج إلى تهيئة:

```
{
  "compilerOptions": {
    "target": "es2022",
    "lib": ["es2022", "esnext.disposable", "dom"]
  }
}
```

مثال:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
  return {
    dispose: () => {
      console.log('Disposing...');
    }
  };
}
```

```

        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is eva

console.log(3);

```

ستطيع الشيفرة ما يلي:

```

1
2
disposed
3

```

Disposable: يجب أن يلتزم المورد المؤهل للتخلص منه بواجهة:

```

// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}

```

عمليات التخلص من الموارد في مكّدس، مما يضمن using تسجل تصريحات التخلص منها بترتيب عكسي لترتيب التصريح:

```

{
    using j = getA(),
        y = getB();
    using k = getC();
} // disposes `C`, then `B`, then `A`.

```

يُضمن التخلص من الموارد حتى إذا وقع استثناء أو جرى تنفيذ شيفرة لاحقة. قد يؤدي هذا إلى احتمال أن تطرح عملية التخلص استثناءً قد يكبت استثناءً آخر. وللاحتفاظ بمعلومات عن الأخطاء المكبوتة، أضيف استثناء أصلي جديد هو `SuppressedError`.

await using تصريح

مع مورد قابل للتخلص منه بصورة غير متزامنة. `await using` يتعامل تصريح وستنتظر نتيجته، `Symbol.asyncDispose` يجب أن تحتوي القيمة على أسلوب عند نهاية الكتلة.

```
async function doWorkAsync() {  
    await using work = doWorkAsync(); // Resource is declared  
} // Resource is disposed (e.g., `await work[Symbol.asyncDispose`
```

لكي يكون المورد قابلاً للتخلص منه بصورة غير متزامنة، يجب أن يلتزم `AsyncDisposable` أو `Disposable` بواجهة:

```
// lib.esnext.disposable.d.ts  
interface AsyncDisposable {  
    [Symbol.asyncDispose](): Promise<void>;  
}  
  
//@ts-ignore  
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // Simple  
  
class DatabaseConnection implements AsyncDisposable {  
    // A method that is called when the object is disposed async  
    [Symbol.asyncDispose]() {  
        // Close the connection and return a promise  
        return this.close();  
    }  
  
    async close() {  
        console.log('Closing the connection...');  
        await new Promise(resolve => setTimeout(resolve, 1000));  
    }  
}
```

```

        console.log('Connection closed.');
```

```

    }
}

async function doWork() {
    // Create a new connection and dispose it asynchronously when
    await using connection = new DatabaseConnection(); // Resource
    console.log('Doing some work...');
} // Resource is disposed (e.g., `await connection[Symbol.asyncDispose]`

doWork();
```

تطبع الشيفرة ما يلي:

```

Doing some work...
Closing the connection...
Connection closed.
```

for و for-in: في العبارات await using يُسمح باستخدام تصريح for و for-of و switch و for-await-of.

سمات الاستيراد

تسميات لعمليات الاستيراد (بيئة) TypeScript 5.3 تخبر سمات الاستيراد في (غيرها). يعزز ذلك الأمان من JSON التشغيل بكيفية التعامل مع الوحدات خلال ضمان وضوح عمليات الاستيراد، ويتوافق مع سياسة أمان المحتوى صلاحيتها، TypeScript من أجل تحميل الموارد بأمان أكبر. تضمن (CSP) لكنها تترك البيئة التشغيل تفسيرها من أجل التعامل المحدد مع الوحدات

مثال:

```
import config from './config.json' with { type: 'json' };
```

مع الاستيراد الديناميكي:

```
const config = import('./config.json', { with: { type: 'json' } })
```

التحقق من صياغة التعبيرات النمطية

من القيم الحرفية TypeScript 5.5.4، تحقق بدءًا من للتعبيرات النمطية بحثًا عن الأخطاء الشائعة في وقت التصريف (مثل الصياغة غير الصالحة والمراجع الخلفية الخاطئة والميزات غير المدعومة في إصدار المستهدف). يساعد ذلك في اكتشاف الأخطاء مبكرًا، لكنه لا JavaScript new RegExp(...).

```
let r = /(a)\2/; // Error: This backreference refers to a group
```

import defer

تحميل وحدة مع تأخير تنفيذها إلى أن تستخدم شيئًا import defer يتيح لك منها فعليًا. يساعد ذلك على تجنب العمل والآثار الجانبية غير الضرورية.

- import defer * as name from "module": يعمل فقط مع
- لا تُنفَّذ الشيفرة إلا عند الوصول إلى أحد التصديرات